SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Luka Pošta

IZRADA TRODIMENZIONALNE VIDEOIGRE BESKONAČNOG TRČANJA U PROGRAMSKOM ALATU UNITY

ZAVRŠNI RAD

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Luka Pošta

Matični broj: 0016154500

Studij: Informacijski i poslovni sustavi

IZRADA TRODIMENZIONALNE VIDEOIGRE BESKONAČNOG TRČANJA U PROGRAMSKOM ALATU UNITY

ZAVRŠNI RAD

Mentor/Mentorica:

Doc. dr. sc. Mladen Konecki

Varaždin, rujan 2025.

Luka Pošta

Izjava o izvornosti

Izjavljujem da je moj završni rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor/Autorica potvrdio/potvrdila prihvaćanjem odredbi u sustavu FOI-radovi

Sažetak

Ovaj rad prikazuje proces izrade računalne igre žanra beskonačnog trčanja u razvojnom okruženju Unity. Cilj je bio osmisliti i realizirati funkcionalni prototip igre koji obuhvaća ključne elemente razvoja: upravljanje igračem, dinamičko oblikovanje prostora igre, sustav prepreka, sustav prikupljanja predmeta, vremensko ograničenje te praćenje rezultata. Posebna pozornost posvećena je korištenju programskog jezika C# i dodatnih komponenti kao što su TextMeshPro i Cinemachine, dok su vizualni i zvučni resursi preuzeti iz besplatnih izvora, ponajprije Unity Asset Storea i platforme OpenGameArt.org.

U radu se analizira arhitektura igre i implementacija pojedinih funkcionalnosti. Naglasak je stavljen na algoritme koji omogućuju kontinuirano generiranje prostora igre, pojavu prepreka i predmeta, obračun rezultata, upravljanje vremenom te prilagodbu težine igre tijekom igranja. Praktični dio rada potvrđuje da je moguće oblikovati zaokruženu računalnu igru oslanjajući se isključivo na besplatne izvore i vlastito programsko rješenje.

Zaključuje se kako razvoj računalnih igara u Unityju predstavlja učinkovit način za stjecanje praktičnih znanja iz programiranja, rada s objektima u trodimenzionalnom prostoru te oblikovanja algoritamskih rješenja. Na kraju rada iznose se ostvareni ciljevi te prijedlozi za buduće nadogradnje igre kroz dodavanje novih sadržaja i funkcionalnosti.

Ključne riječi: Unity; C#; endless runner; računalne igre; dinamičko generiranje prostora; prepreke; pickup; bodovanje

Sadržaj

Sadržaj iii

1. Uvod 1

2. Metode i tehnike rada 2

2.1. Metodološki pristup i aktivnosti tijekom izrade 2

2.2. Korišteni alati i resursi 3

3. Razrada teme 6

3.1. Žanr beskonačnog trčanja 6

3.1.1. Povijesni pregled i značaj žanra 6

3.1.2. Primjeri poznatih igara 7

3.1.3. Ključna obilježja žanra 7

3.2. Opis razvijene igre 8

3.2.1. Pravila i ciljevi igre 9

3.2.2. Lik igrača i kontrole 10

3.2.3. Okruženje i vizualni identitet 11

3.2.4. Sustav prepreka i izazova 12

3.2.5. Predmeti i kontrolne točke 14

3.2.6. Korisničko sučelje (rezultat, vrijeme, poruke) 15

3.3. Struktura sustava igre 17

3.3.1. Organizacija projekta u razvojnom okruženju 17

3.3.2. Struktura scena (glavni izbornik i prostor igre) 18

3.3.3. Uloga pojedinih funkcionalnih cjelina 19

3.4. Implementacija funkcionalnosti 20

3.4.1. Upravljanje igračem i sustav sudara 21

3.4.2. Dinamičko oblikovanje prostora igre 23

3.4.3. Praćenje bodova i vremena 24

3.4.4. Dinamičko upravljanje kamerom 25

3.4.5. Glazbena podloga 26
3.4.6. Navigacija kroz izbornike 27
3.5. Algoritamska rješenja 28
3.5.1. Sustav bodovanja 29
3.5.1.1. Logika bodovanja 29
3.5.1.2. Povezanost s korisničkim sučeljem 32
3.5.2. Sustav mjerenja vremena 33
3.5.2.1. Početno odbrojavanje 34
3.5.2.2. Produženje vremena 35
3.5.2.3. Završetak igre 37
3.5.3. Kontrola težine igre 38
4. Zaključak 41
Popis literature 42
Popis slika 43

1. Uvod

Razvoj računalnih igara predstavlja jedno od najdinamičnijih područja suvremenih informacijskih tehnologija. Igre se koriste u različite svrhe, od zabave i razonode do edukacije i simulacija. Poseban segment industrije čine tzv. igre beskonačnog trčanja (eng. *endless runner*), koje se odlikuju jednostavnim pravilima, stalnim generiranjem sadržaja i stalnim izazovima za igrača. Njihova popularnost temelji se na brzoj dinamici, mogućnosti postizanja visokih rezultata te motivaciji za ponavljanje igre.

Cilj ovog rada je prikazati proces izrade računalne igre žanra endless runner, izrađene u razvojnom okruženju Unity uz korištenje programskog jezika C#. Kroz rad će se opisati korišteni alati, struktura igre, način implementacije ključnih funkcionalnosti te algoritmi koji omogućuju generiranje levela, upravljanje vremenom, bodovanje i druge mehanike.

Praktični dio rada obuhvaća funkcionalni prototip igre u kojem igrač preuzima ulogu lika koji se kreće beskonačnim prostorom, izbjegava prepreke i prikuplja predmete kako bi produžio trajanje igre ili ostvario veći broj bodova. Razvoj igre temelji se na kombinaciji postojećih resursa dostupnih u Unity Asset Storeu te vlastitih skripti izrađenih u C#.

Svrha rada je prikazati postupak razvoja jednostavne, ali kompletne igre koja u sebi objedinjuje osnovne elemente razvoja računalnih igara: kreiranje likova i okruženja, implementaciju kontrole igrača, sustava prepreka, sustava prikupljanja predmeta, vremenskog ograničenja i završetka igre. Osim toga, rad pokazuje kako se može postići funkcionalna i zaokružena aplikacija uz korištenje dostupnih besplatnih resursa i odgovarajućih alata.

Struktura rada organizirana je na sljedeći način. U drugom poglavlju opisane su metode i tehnike korištene pri izradi igre, uključujući programske alate i izvore korištenih resursa. Treće poglavlje donosi detaljnu razradu teme, s naglaskom na opis igre, njene strukture, implementiranih funkcionalnosti i algoritama. U četvrtom poglavlju je iznesen zaključak s pregledom ostvarenih ciljeva i prijedlozima za daljnji razvoj.

2. Metode i tehnike rada

Izrada računalne igre zahtijeva jasno definiran metodološki okvir, pažljivo planirane aktivnosti i odabir odgovarajućih alata. U ovom poglavlju opisuje se pristup korišten pri razvoju igre, tijek provedbe aktivnosti te alati i resursi kojima je ostvarena praktična realizacija projekta.

2.1. Metodološki pristup i aktivnosti tijekom izrade

Metodološki okvir rada temelji se na projektno-implementacijskom pristupu provedenom kroz niz jasno definiranih faza. Svaka faza bila je usmjerena na određeni segment razvoja igre, a njihov slijed i međusobno nadograđivanje rezultirali su funkcionalnim prototipom. Proces je proveden iterativno, što znači da je svaka funkcionalnost najprije implementirana u osnovnom obliku, zatim testirana i postupno unaprjeđivana radi stabilnosti sustava.

Prva faza obuhvatila je analizu i planiranje. Proučen je žanr beskonačnog trčanja te su identificirani temeljni mehanizmi. Na temelju toga definirani su ciljevi, pravila igranja i osnovna scena s definiranim likom igrača i parametrima prostora igre.

U drugoj fazi oblikovana je struktura sustava i prostor igre koji se kontinuirano nadograđuje ispred igrača, dok se prethodni dijelovi uklanjaju. Time je ostvareno beskonačno trajanje igre i stalna nepredvidivost okruženja.

Treća faza bila je usmjerena na implementaciju osnovnih mehanizama. Uvedeno je kretanje igrača i precizno upravljanje s ograničenjima koja sprječavaju napuštanje predviđenih granica. Istodobno je razvijen sustav detekcije sudara i reakcija na prepreke, čime je oblikovana temeljna mehanika igranja i osnovni izazov.

U četvrtoj fazi uvedene su proširene funkcionalnosti. Dodane su prepreke, predmeti za prikupljanje te sustavi bodovanja i vremenskog ograničenja. Predmeti su imali različite učinke, uključujući povećavanje broja osvojenih bodova, privremenu promjenu brzine kretanja i produženje vremena igranja. Time je postignuta veća raznolikost, dinamičnost i jasna struktura ciljeva.

Peta faza odnosila se na završnu doradu i testiranje. Dodani su vizualni i zvučni elementi, oblikovano je korisničko sučelje te razvijen osnovni izbornik s prikazom rezultata i vremena, mogućnošću pokretanja nove igre te povratkom u glavni izbornik ili izlaskom iz

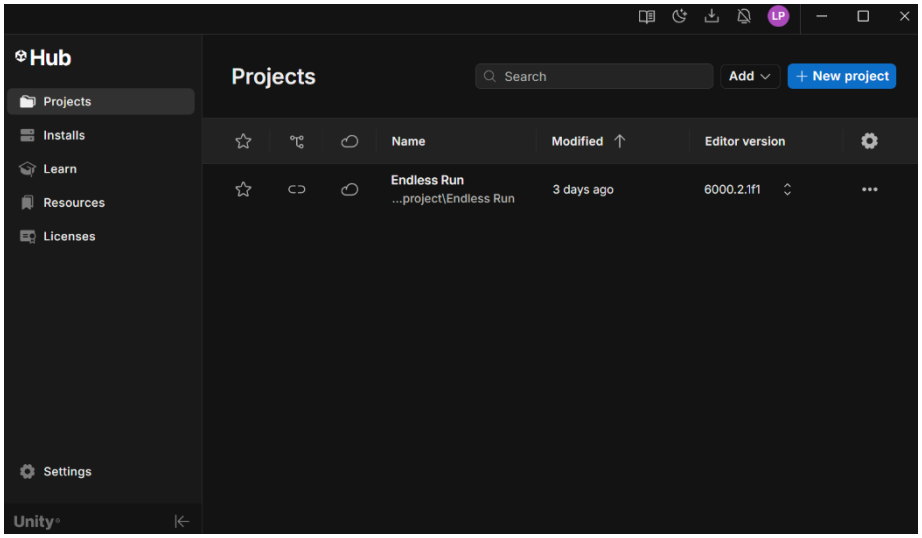
aplikacije. Provedena su ponovljena testiranja radi otkrivanja i otklanjanja pogrešaka te provjere stabilnosti sustava.

Cjelokupan proces, od planiranja do dorade, omogućio je sustavno unaprjeđivanje i zaokruživanje igre u funkcionalni prototip.

2.2. Korišteni alati i resursi

Za realizaciju završnog rada korišten je niz programsko-razvojnih alata i besplatnih resursa koji su zajednički omogućili izradu funkcionalne računalne igre. Svaki od alata imao je specifičnu ulogu u procesu razvoja, a njihova je kombinacija pridonijela uspješnom ostvarivanju zadanih ciljeva.

Unity predstavlja središnje razvojno okruženje korišteno za izradu igre. Riječ je o jednom od najraširenijih alata u industriji računalnih igara, poznatom po jednostavnom sučelju i širokom rasponu funkcionalnosti. Unity je omogućio oblikovanje trodimenzionalnog prostora, smještanje objekata unutar scene te jednostavnu integraciju vizualnih i zvučnih resursa. Njegova fleksibilnost očitovala se u mogućnosti izvođenja igre na različitim platformama, a dodatna prednost bila je velika zajednica korisnika i dostupna dokumentacija.



Slika 1: Unity Hub sučelje

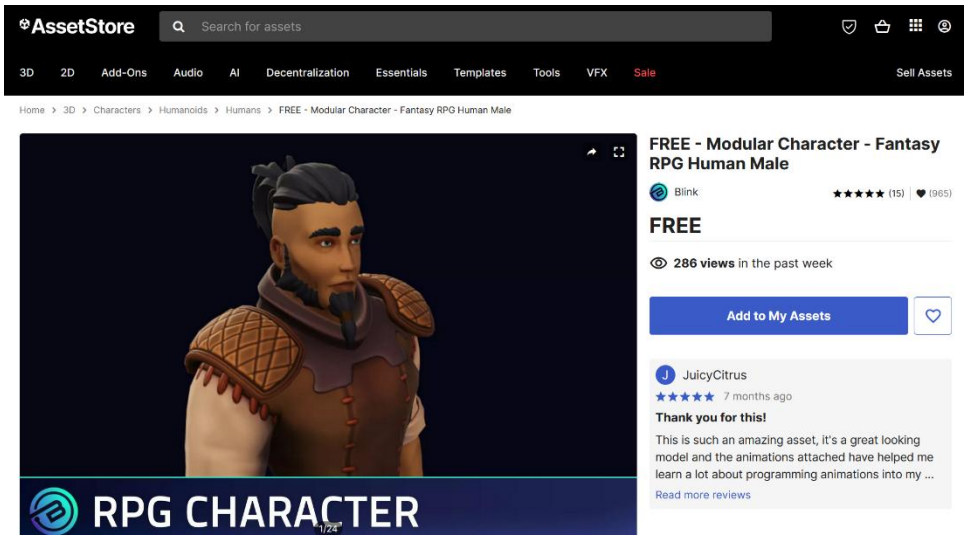
C# je korišten kao glavni programski jezik za definiranje ponašanja objekata i implementaciju logike igre. Njegova povezanost s Unityjem omogućila je jednostavnu integraciju skripti i modularno upravljanje funkcionalnostima. Za izradu koda korišten je Visual

Studio, integrirano razvojno okruženje koje nudi preglednu strukturu projekta, isticanje pogrešaka, automatsko dovršavanje koda i alate za otklanjanje pogrešaka. Upravo je kombinacija jezika C# i okruženja Visual Studio pridonijela brzini i preciznosti implementacije.

TextMeshPro poslužio je za oblikovanje tekstualnih elemenata korisničkog sučelja. U usporedbi s osnovnim tekstualnim komponentama koje nudi Unity, TextMeshPro pruža kvalitetniji prikaz, veću čitljivost i brojne mogućnosti oblikovanja. Time je osigurano da prikaz rezultata, vremena i drugih informacija bude jasan i estetski usklađen s vizualnim identitetom igre.

Cinemachine je korišten za dinamičko upravljanje kamerom. Ovaj alat omogućuje prilagodbu kuta snimanja i vidnog polja tijekom igre, čime se naglašavaju promjene u brzini i intenzitetu kretanja. Njegova uporaba značajno je poboljšala vizualni doživljaj i pridonijela uronjenosti igrača u samu igru.

Osim programskih alata, korišteni su i besplatni resursi dostupni putem interneta. **Unity Asset Store** poslužio je kao izvor vizualnih elemenata, uključujući modele likova, prepreka i predmeta. Odabrani elementi preuzeti su u besplatnim inačicama, što je omogućilo oblikovanje igre bez dodatnih troškova. Na taj je način ostvarena ravnoteža između estetske kvalitete i dostupnosti.



Slika 2: Unity Asset Store

Za zvučnu podlogu korištena je platforma **OpenGameArt.org**, koja nudi glazbu i zvučne efekte pod slobodnim licencama. Odabrana glazba reproducira se u pozadini tijekom igranja i doprinosi stvaranju atmosfere i dinamike igre. Zahvaljujući toj platformi omogućeno je uključivanje kvalitetnog zvučnog sadržaja uz poštivanje pravnih okvira.

Kombinacija navedenih alata i resursa pokazala se kao optimalna za izradu funkcionalnog prototipa igre. S jedne strane, korištena su provjerena profesionalna razvojna okruženja i biblioteke, a s druge strane besplatni vizualni i zvučni elementi koji su omogućili dovršetak projekta unutar zadanih okvira.

3. Razrada teme

U ovom poglavlju prikazuje se središnji dio rada u kojem se detaljno razmatra proces izrade igre. Najprije se objašnjavaju obilježja žanra beskonačnog trčanja i njegova važnost u suvremenoj industriji računalnih igara. Nakon toga slijedi opis razvijene igre, njezinih pravila i osnovnih mehanizama, kao i strukture sustava kojom je realizirana. Posebna se pažnja posvećuje tehničkoj implementaciji i algoritmima koji omogućuju funkcioniranje igre, a prikazani su i odabrani primjeri koda i vizualnih rješenja.

3.1. Žanr beskonačnog trčanja

Jedan od najprepoznatljivijih i najraširenijih žanrova u području računalnih igara posljednjih desetljeća jest žanr beskonačnog trčanja. Riječ je o igrama koje se odlikuju neprekidnim kretanjem lika kroz prostor, stalnim izazovima i jednostavnim pravilima koja omogućuju brzo razumijevanje i široku pristupačnost. Popularnost ovoga žanra povezana je s razvojem mobilnih uređaja i potrebom za kratkim, ali intenzivnim iskustvima igranja.

Kako bi se potpunije razumio značaj ovoga žanra, u nastavku će se prikazati njegov povijesni razvoj, primjeri najpoznatijih igara te temeljna obilježja koja ga čine prepoznatljivim.

3.1.1. Povijesni pregled i značaj žanra

Počeci žanra beskonačnog trčanja sežu u kasne 2000-te godine, kada su se pojavile prve igre koje su eksperimentirale s konceptom neprekidnog kretanja. U početku su to bile jednostavne dvodimenzionalne igre razvijene za mrežne preglednike, često u obliku kratkih zabavnih aplikacija dostupnih besplatno. Njihova privlačnost temeljila se na brzom ulasku u igru i kratkom vremenu potrebnom za jedno igranje, što ih je činilo prikladnima za kraće pauze i opuštanje.

Pravi procvat ovoga žanra dogodio se s pojavom pametnih telefona i trgovina mobilnih aplikacija. Igra *Temple Run*, objavljena 2011. godine, uvelike je popularizirala koncept i postavila temelje za kasnije naslove. Ubrzo su uslijedile brojne inačice i igre sličnog tipa, među kojima se ističe *Subway Surfers* kao jedna od najpreuzimanijih igara na mobilnim platformama uopće.

Značaj žanra beskonačnog trčanja očituje se u njegovoj širokoj prihvaćenosti. Iako se radi o jednostavnom konceptu, ovakve igre privukle su milijune korisnika različite dobi i iskustva. Njihova popularnost leži u kombinaciji stalnog izazova, mogućnosti postizanja visokih rezultata i kratkog trajanja pojedinačne igre, što ih čini idealnima za moderni način života obilježen brzinom i mobilnošću.

3.1.2. Primjeri poznatih igara

Razvoj i popularnost žanra beskonačnog trčanja najbolje se mogu prepoznati kroz nekoliko naslova koji su ostvarili globalni uspjeh. Među njima se ističu već spomenute igre *Temple Run* i *Subway Surfers*, koje su obilježile početak šireg prihvaćanja ovoga tipa igara na mobilnim uređajima.

Temple Run, objavljen 2011. godine, uveo je trodimenzionalno okruženje u kojem igrač neprekidno bježi kroz prostor ispunjen preprekama i zamkama. Dinamično upravljanje pokretima i stalna prijetnja od gubitka igre stvorili su napetost koja je privukla velik broj korisnika. Igra je vrlo brzo postala jedna od najprepoznatljivijih na mobilnim platformama.

Subway Surfers, objavljen 2012. godine, dodatno je popularizirao žanr uvođenjem šarenog vizualnog stila i jednostavne mehanike prilagođene širokoj publici. Posebnu je važnost imala činjenica da je igra bila besplatna za preuzimanje, što je doprinijelo njezinoj masovnoj rasprostranjenosti i održavanju popularnosti kroz stalna ažuriranja i dodavanje novih sadržaja.

Osim navedenih, spomena vrijedni su i drugi naslovi poput *Minion Rush* i *Jetpack Joyride*, koji su dodatno obogatili ponudu i pokazali kako se osnovni koncept beskonačnog trčanja može prilagoditi različitim temama i vizualnim stilovima.

3.1.3. Ključna obilježja žanra

Unatoč različitim vizualnim rješenjima i tematskim okruženjima, igre žanra beskonačnog trčanja dijele niz zajedničkih obilježja koja ih čine lako prepoznatljivima i definiraju njihovu strukturu.

Prvo i osnovno obilježje jest **kontinuirano kretanje lika** kroz prostor. Za razliku od igara s unaprijed određenim razinama i krajnjim ciljem, ovdje ne postoji jasno definiran završetak. Lik se kreće beskonačno prema naprijed, a zadatak igrača jest održavati to kretanje

što je duže moguće. Na taj način naglasak se stavlja na trajanje igre i sposobnost igrača da se prilagođava novim izazovima.

Drugo ključno obilježje predstavlja progresivno **povećavanje težine**. Kako igra odmiče, izazovi postaju zahtjevniji – povećava se brzina kretanja, pojavljuju se složenije prepreke ili se pred igrač postavljaju dodatni zadaci. Ovaj element osigurava da igra ne postane monotona te da se tijekom vremena održava napetost i motivacija.

Treći element čini sustav **nagrađivanja i bodovanja**. Igrač se potiče na postizanje što boljeg rezultata, najčešće mjerenog brojem prijeđenih jedinica udaljenosti, prikupljenim predmetima ili kombinacijom različitih čimbenika. Rezultat se prikazuje na kraju svake igre, što motivira na ponovno igranje i pokušaj postizanja boljeg uspjeha.

Četvrto obilježje odnosi se na **jednostavnost upravljanja**. Upravo ta jednostavnost omogućuje široku dostupnost žanra. Igrači mogu odmah započeti s igranjem bez potrebe za dugotrajnim učenjem pravila. Upravljačke naredbe svode se na pomicanje u nekoliko smjerova, skakanje ili izbjegavanje prepreka, što je dovoljno da se ostvari bogato i uzbudljivo iskustvo igranja.

Konačno, značajno obilježje žanra jest njegova visoka razina **ponovljivosti**. Budući da igra nema konačan završetak, svako novo igranje donosi drukčiji niz prepreka i izazova. Time se osigurava stalna neizvjesnost, a upravo ta nepredvidivost jedan je od glavnih razloga dugotrajne popularnosti ovih igara.

Sva navedena obilježja zajedno čine jezgru žanra beskonačnog trčanja i objašnjavaju njegovu široku rasprostranjenost i privlačnost.

3.2. Opis razvijene igre

Na temelju obilježja žanra beskonačnog trčanja izrađena je vlastita računalna igra koja u sebi objedinjuje osnovne elemente tog tipa. Igra je osmišljena kao funkcionalni prototip čiji je cilj prikazati kako se jednostavnim pravilima i ograničenim resursima može oblikovati cjelovito iskustvo igranja.

U ovoj cjelini opisani su pravila i ciljevi igre, lik kojim igrač upravlja, vizualno okruženje i identitet, sustav prepreka, predmeti koje je moguće prikupljati te korisničko sučelje. Svaka od navedenih sastavnica pridonosi cjelovitosti igre i zajednički čine logički zaokružen sustav.

3.2.1. Pravila i ciljevi igre

Razvijena igra temelji se na jasnim pravilima i jednostavnoj strukturi koja igraču omogućuje brzo snalaženje, ali istodobno nudi dovoljno izazova da zadrži njegovu pažnju kroz dulje vrijeme igranja.

Središnje pravilo igre jest stalno **kretanje lika** prema naprijed u trodimenzionalnom prostoru. Igrač nema mogućnost zaustavljanja lika, već se njegovo kretanje odvija neprekidno i stalnim tempom. Uloga igrača svodi se na upravljanje smjerom kretanja kako bi se izbjegle prepreke i prikupili korisni predmeti. Time se ostvaruje dojam dinamičnosti, a igra postaje izazov koji zahtijeva brze reakcije i stalnu usredotočenost.

Slika 3: Sučelje igre

Drugi važan element pravila igre odnosi se na **prepreke**. One se pojavljuju na različitim pozicijama unutar prostora, a igrač ih mora zaobići preciznim upravljanjem. Sudar s preprekom rezultira gubitkom kontrole i završetkom igre. Prepreke stoga predstavljaju ključan izvor izazova, jer igrač neprestano mora prilagođavati svoje pokrete i predviđati nadolazeće zapreke.

Pored prepreka, igra uključuje i **predmete** koje je moguće prikupljati. Svaki tip predmeta ima poseban učinak na tijek igranja. Određeni predmeti povećavaju ukupan broj bodova, čime se nagrađuje uspješno snalaženje i pruža igraču dodatna motivacija. Drugi tipovi predmeta utječu na brzinu kretanja – neki je privremeno povećavaju, što igru čini napetijom i zahtjevnijom, dok drugi usporavaju kretanje i tako igraču daju više vremena za reakciju. Posebnu ulogu imaju predmeti koji produžuju preostalo vrijeme igranja, čime omogućuju dulje trajanje i ostvarivanje boljeg rezultata.

Vrijeme je treći ključni čimbenik u pravilima igre. Na početku svakog igranja postavljeno je određeno trajanje, a istek vremena označava kraj igre neovisno o ostalim čimbenicima. Upravo taj vremenski okvir dodatno pojačava osjećaj napetosti i potiče igrača da što učinkovitije iskoristi svaki trenutak.

Ciljevi igre proizlaze iz navedenih pravila. Primarni cilj jest ostati u igri što je dulje moguće, pritom izbjegavajući prepreke i prikupljajući korisne predmete. Sekundarni cilj odnosi se na postizanje što većeg broja bodova, koji se prikazuju na završnom ekranu i služe kao mjerilo uspješnosti. Takav sustav bodovanja uvodi natjecateljski element, jer igrač može nastojati nadmašiti vlastite prethodne rezultate ili rezultate drugih igrača.

Kombinacija jednostavnih pravila i jasno postavljenih ciljeva čini igru pristupačnom širokom krugu korisnika, ali istodobno dovoljno izazovnom da potakne višekratno igranje. U tom se balansu između dostupnosti i izazova prepoznaje temeljna vrijednost razvijenog prototipa.

3.2.2. Lik igrača i kontrole

Središnji element svake igre jest lik kojim igrač upravlja. U razvijenom prototipu odabran je humanoidni lik preuzet iz besplatnih izvora dostupnih putem Unity Asset Storea. Takav odabir omogućio je brzo oblikovanje igračkog lika bez potrebe za izradom vlastitih trodimenzionalnih modela, dok je fokus mogao ostati na implementaciji mehanike i logike igre. Lik je oblikovan tako da bude prepoznatljiv i jasno uočljiv unutar prostora igre, a pritom dovoljno neutralan da se može uklopiti u različite kontekste ili buduće nadogradnje.

Slika 4: Model trkača kojim igrač upravlja

Kontrole su osmišljene tako da budu intuitivne i jednostavne, čime se osigurava da igrač odmah može započeti s igranjem bez prethodnog učenja složenih kombinacija tipki. Kretanje lika ostvaruje se po unaprijed određenim osima, a igrač ima mogućnost pomicanja ulijevo i udesno kako bi izbjegavao prepreke, kao i pomicanja naprijed i nazad. Ova kombinacija osnovnih pokreta dovoljna je za stvaranje raznolikih izazova tijekom igre.

Posebna pažnja posvećena je uravnoteženju osjetljivosti kontrola. Brzina reakcije lika na korisničke naredbe mora biti dovoljno visoka da igrač osjeti izravnu povezanost svojih postupaka i događaja na ekranu, ali ne smije biti pretjerana kako ne bi otežala precizno upravljanje. Upravo ta ravnoteža čini razliku između igre koja je frustrirajuća i one koja potiče na višekratno igranje.

Lik igrača **nije statičan** element već sudjeluje u animacijama koje dodatno obogaćuju iskustvo igranja. Tijekom kretanja vidljive su osnovne animacije trčanja, dok se pri interakciji s preprekama aktiviraju posebne animacije koje naglašavaju posljedice sudara. Time se postiže veći stupanj realističnosti i dinamike, a igrač dobiva jasnu vizualnu povratnu informaciju o svojim postupcima.

Kombinacija odabranog modela, jednostavnih i intuitivnih kontrola te animacijskih elemenata stvara uravnotežen sustav kojim je oblikovan temeljni doživljaj igre. Upravo je lik igrača poveznica između korisnika i virtualnog prostora te osnovni kanal putem kojega se ostvaruje interakcija i uranjanje u igru.

3.2.3. Okruženje i vizualni identitet

Vizualni dojam igre u velikoj mjeri oblikuje iskustvo igranja i pridonosi stvaranju atmosfere koja privlači i zadržava pažnju igrača. U razvijenom prototipu okruženje je zamišljeno kao beskonačni put kojim se lik stalno kreće prema naprijed. Prostor se sastoji od niza međusobno povezanih segmenata koji se dinamički pojavljuju ispred igrača, dok se prethodni dijelovi uklanjaju. Na taj način stvara se dojam kontinuiranog kretanja kroz beskonačan svijet.

Svi elementi okruženja preuzeti su iz besplatnih izvora dostupnih na Unity Asset Storeu. Riječ je o trodimenzionalnim modelima koji prikazuju staze, ograde i različite prostorne strukture. Iako nisu originalno izrađeni za potrebe ovog rada, njihova primjena omogućila je oblikovanje dovoljno uvjerljivog vizualnog identiteta igre, a pritom je razvoj ostao usmjeren na implementaciju mehanike i algoritama.

Vizualni identitet igre temelji se na jednostavnosti i preglednosti. Boje i kontrasti odabrani su tako da jasno naglašavaju razliku između prostora, prepreka i predmeta koje igrač

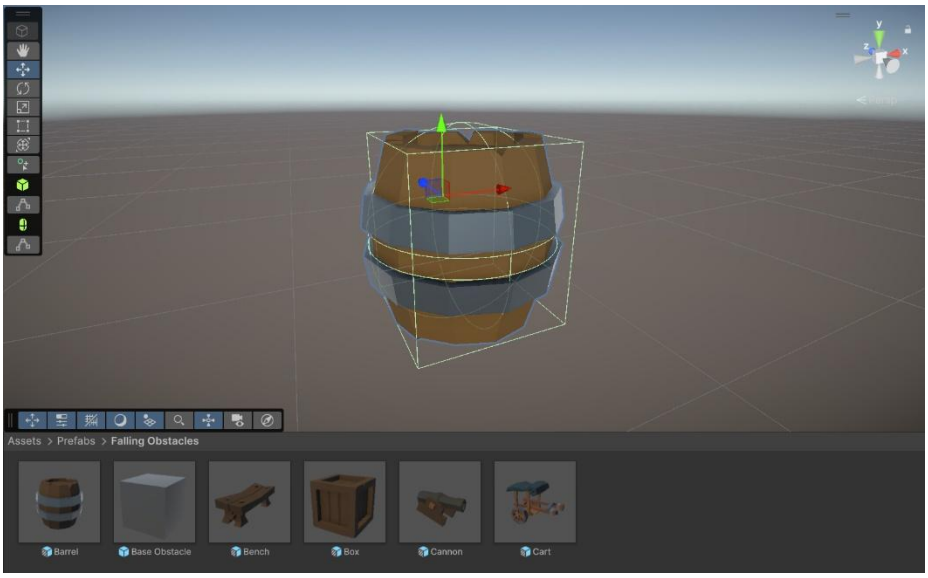
prikuplja. Time je postignuto da igrača ne zbunjuju nepotrebni detalji, već da mu pozornost ostane usmjerena na ključne zadatke.

Posebnu ulogu ima i kamera koja prati kretanje lika. Zahvaljujući dodatnoj komponenti Cinemachine, kamera se prilagođava promjenama u brzini i dinamici igre, čime se naglašava doživljaj napetosti. Lagane promjene u kutu gledanja i vidnom polju stvaraju dojam ubrzanja ili usporavanja, a to vizualno obogaćuje igru i čini je privlačnijom.

Ukupno gledano, okruženje i vizualni identitet igre oblikovani su tako da pruže funkcionalnu i estetski skladnu pozadinu osnovnim mehanikama igranja. Iako je riječ o prototipu, postavljena je čvrsta osnova koja omogućuje daljnje nadogradnje u smjeru detaljnijeg i vizualno bogatijeg svijeta.

3.2.4. Sustav prepreka i izazova

Jedan od ključnih elemenata žanra beskonačnog trčanja jest prisutnost **prepreka** (klupe, topovi, kolica, bačve, kutije, ograde, …) koje igrač mora izbjegavati. Upravo one stvaraju osjećaj napetosti i održavaju dinamiku igre. Bez prepreka, kretanje bi bilo monotono, a igrač bi izgubio interes nakon kratkog vremena.



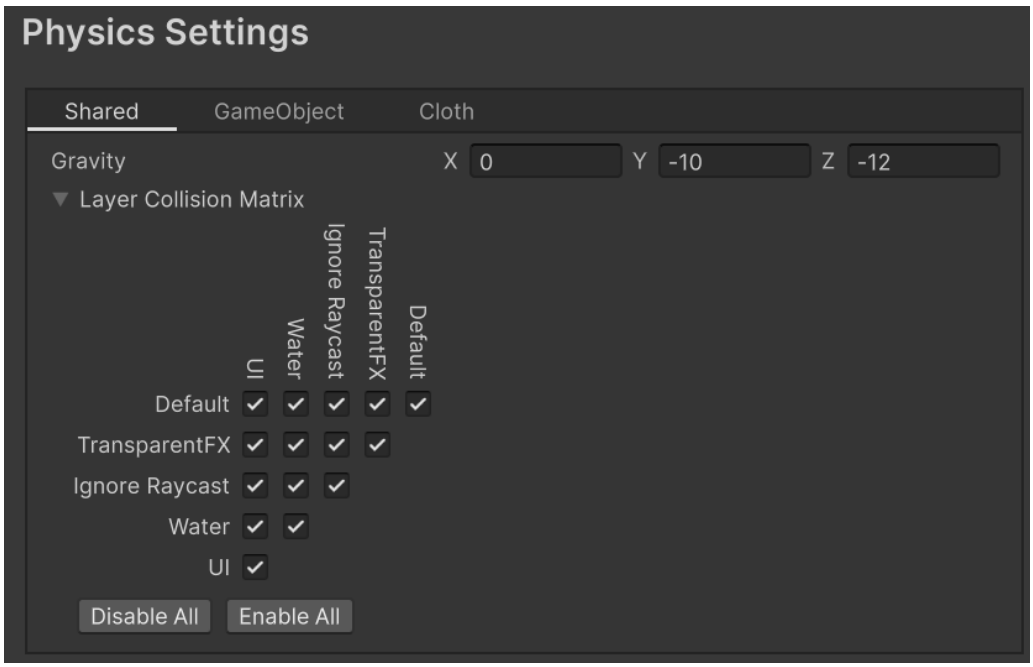
Slika 5: Modeli prepreka

U razvijenom prototipu prepreke su zamišljene kao objekti koji se nasumično pojavljuju na putu igrača. Njihova uloga jest prisiliti igrača na brze reakcije i precizno upravljanje. Pritom se ne radi o jednoličnom nizu prepreka, već se one postavljaju u različitim položajima i

kombinacijama, čime se stvara stalna nepredvidivost. Na taj način igra ne postaje predvidiva, već zahtijeva stalnu usredotočenost.

Svaka prepreka u igri opremljena je **BoxCollider** komponentom koja definira njezin kolizijski obujam. Na taj način igra prepoznaje sudar između igrača i prepreke, čime se aktivira odgovarajuća logika, primjerice usporavanje ili gubitak vremena. Za statične prepreke koristi se kombinacija BoxCollidera i zaključanog **Rigidbodyja** kako bi ostale na mjestu, ali i dalje pravilno reagirale na kontakt s igračem.

Za nadolazeće prepreke koristi se fizički sustav Unityja temeljen na unaprijed definiranim vrijednostima gravitacije i matrici sudara (eng. *Layer Collision Matrix*). U ovom projektu gravitacija je prilagođena tako da ima komponente Y = -10 i Z = -12, čime se postiže prirodan pad objekata prema dolje, ali i njihovo dodatno gibanje prema igraču. Time prepreke ne ostaju statične, nego se konstantno kreću prema točki sudara. Matrica sudara određuje koje će se slojeve objekata međusobno detektirati, čime se postiže da prepreke reagiraju samo na relevantne objekte u igri (npr. igrača), dok se interakcije s elementima korisničkog sučelja ili pomoćnim slojevima ignoriraju.



Slika 6: Fizičke postavke za nadolazeće prepreke

Sudari s preprekama imaju izravne posljedice. Kada dođe do sudara, igra registrira pogrešku i igrač gubi mogućnost nastavka kretanja. Ovaj mehanizam čini prepreke glavnim izazovom igre jer jedan pogrešan potez može označiti kraj. Osim funkcionalnog aspekta,

prepreke su vizualno oblikovane tako da budu jasno prepoznatljive, ali se ipak skladno uklapaju u cjelokupni izgled okruženja.

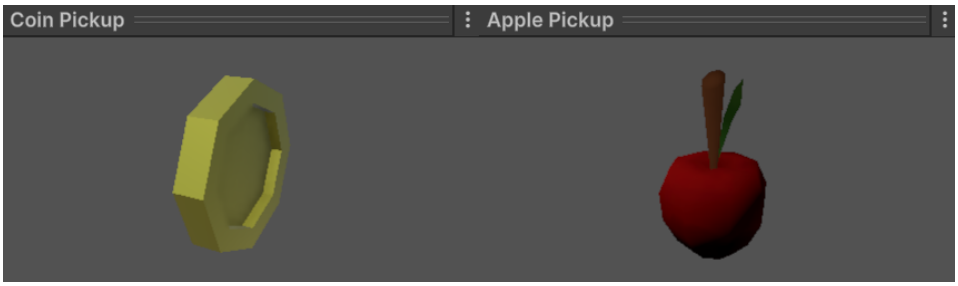
Kako bi igra ostala zanimljiva tijekom duljeg igranja, prepreke se pojavljuju u različitim ritmovima i s promjenjivim razmacima. Time se sprječava učenje unaprijed određenih obrazaca i održava se osjećaj napetosti. Uz to, uvođenjem postupnog povećavanja brzine kretanja, prepreke postaju sve teže za izbjegavanje, čime se prirodno povećava razina izazova. Sustav prepreka u ovom prototipu pokazuje kako relativno jednostavan koncept može osigurati visoku razinu dinamičnosti i dugotrajnu motivaciju za igru.

3.2.5. Predmeti i kontrolne točke

Osim prepreka, koje igrač mora izbjegavati, važnu ulogu u igri imaju i predmeti te kontrolne točke. Oni ne služe samo kao dodatni vizualni elementi, već imaju izravnu funkcionalnu svrhu i znatno utječu na tijek igranja. Njihova prisutnost uvodi raznolikost, povećava dinamičnost te potiče igrača na donošenje odluka u realnom vremenu.

Prva skupina predmeta (zlatnici) povezana je s **bodovanjem**. Njihovim prikupljanjem igrač postupno povećava ukupan broj osvojenih bodova. Bodovi se bilježe tijekom igre i prikazuju na završnom ekranu nakon što igra završi. Time bodovni sustav postaje glavni kriterij uspješnosti, a rezultat ima natjecateljski karakter jer igrač nastoji nadmašiti vlastite prethodne uspjehe ili uspjehe drugih. Upravo ovakav sustav bodovanja čini osnovu motivacije za ponovno igranje, budući da se rezultat uvijek može poboljšati.

Druga skupina predmeta utječe na **brzinu kretanja lika**. Određeni predmeti (jabuke) ubrzavaju tijek igre, čime se povećava razina izazova jer igrač mora reagirati brže i preciznije. Drugi pak predmeti, tj. već spomenute prepreke usporavaju kretanje i tako igraču pružaju privremeni predah. Ova mehanika stvara kontrast između napetosti i olakšanja, čineći igru dinamičnijom. Ujedno prisiljava igrača da promišlja o posljedicama prikupljenih predmeta – iako neki predmeti donose prednost, u isto vrijeme mogu povećati rizik i dovesti do pogreške.



Slika 7: Pickupi (zlatnik i jabuka)

Treći element predstavljaju **kontrolne točke**. One se razlikuju od klasičnih predmeta jer nisu objekti koje igrač „pokupi“, već prostorne cjeline kroz koje prolazi tijekom kretanja. Njihova je funkcija produžiti raspoloživo vrijeme igranja. Time se u igru uvodi još jedan sloj strategije, budući da igrač ne ovisi samo o vlastitoj spretnosti u izbjegavanju prepreka, nego mora pravovremeno doseći kontrolne točke kako bi mogao nastaviti igrati. Kontrolne točke također služe kao svojevrsni ritmički element igre, jer unose povremene trenutke olakšanja i nagrađuju ustrajnost.

Slika 8: Kontrolna točka

Raspored predmeta i kontrolnih točaka unutar igre osmišljen je tako da bude nepredvidiv. Njihova pojava nije uvijek jednaka, već se izmjenjuje u različitim kombinacijama. Time se izbjegava monotonija i osigurava da svako novo igranje donese drukčiji doživljaj. Igrač stoga mora ostati stalno koncentriran, jer ne može unaprijed znati kada će se pojaviti korisni predmeti ili gdje će se nalaziti sljedeća kontrolna točka.

Ukupno gledano, predmeti i kontrolne točke nisu sporedni elementi igre, već čine jedan od njenih temeljnih mehanizama. Oni pružaju dodatnu motivaciju, produljuju trajanje igre i oblikuju osjećaj stalne napetosti. Na taj način igra postaje bogatija i zanimljivija, a jednostavan koncept beskonačnog kretanja nadopunjuje se slojem nagrađivanja i strategije.

3.2.6. Korisničko sučelje (rezultat, vrijeme, poruke)

Korisničko sučelje predstavlja važan posrednički sloj između igrača i sustava. Njegova je funkcija prikazati sve ključne informacije na jasan i pregledan način, bez narušavanja

doživljaja igranja. U razvijenoj igri korisničko sučelje oblikovano je tako da bude minimalističko, ali funkcionalno, uz naglasak na tri glavna elementa: prikaz rezultata, prikaz preostalog vremena i obavijesti koje igrač prima tijekom igre.

Prvi i najistaknutiji element jest prikaz **rezultata**. Rezultat se povećava prikupljanjem predmeta te se u svakom trenutku prikazuje na ekranu. Smješten je na vidljivom mjestu, ali tako da ne ometa pregled prostora igre. Bodovni sustav služi kao stalna motivacija, budući da igrač neprekidno prati vlastiti napredak i nastoji ostvariti što bolji ishod. Rezultat je ujedno i glavni pokazatelj uspješnosti, jer se na kraju svake igre sažeto prikazuje ukupni broj osvojenih bodova.

Drugi element sučelja jest prikaz **vremena**. Budući da je igra vremenski ograničena, jasno isticanje preostalog vremena ključno je za donošenje odluka. Vrijeme se prikazuje u decimalnom obliku, što omogućuje precizno praćenje preostalih sekundi. Kada se vrijeme približi kraju, igrač osjeća dodatnu napetost, a sama mehanika igre postaje zahtjevnija jer se odluke moraju donositi brže. Ako igrač dosegne kontrolnu točku, vrijeme se automatski produžuje, a promjena se odmah odražava na sučelju.

Treći element čine **poruke i obavijesti** koje se prikazuju u ključnim trenucima. Najvažnija među njima jest poruka o završetku igre (eng. *Game Over*), koja se pojavljuje kada istekne vrijeme ili kada dođe do sudara s preprekom. Pojava ove poruke jasno označava završetak sesije igranja i poziva igrača na ponovno započinjanje. Osim toga, sučelje može prikazivati i druge obavijesti, primjerice kratke informacije o učincima prikupljenih predmeta ili vizualne signale koji naglašavaju promjenu brzine kretanja.

Slika 9: Poruka *Game Over*

Posebna pažnja posvećena je oblikovanju teksta unutar sučelja. Korištena je komponenta TextMeshPro, koja omogućuje veću preciznost i estetsku kvalitetu prikaza u usporedbi s osnovnim Unity tekstualnim rješenjima. Time je postignuto da svi elementi budu čitljivi, prilagođeni različitim veličinama zaslona i vizualno usklađeni s cjelokupnim identitetom igre.

Ukupni dojam korisničkog sučelja jest jednostavnost i preglednost. Svi elementi smješteni su tako da igrač u svakom trenutku ima pristup najvažnijim informacijama, ali bez narušavanja dinamike i koncentracije. Na taj način sučelje nadopunjuje osnovnu mehaniku igre i omogućuje igraču da donosi odluke na temelju jasnih i pravovremenih informacija.

3.3. Struktura sustava igre

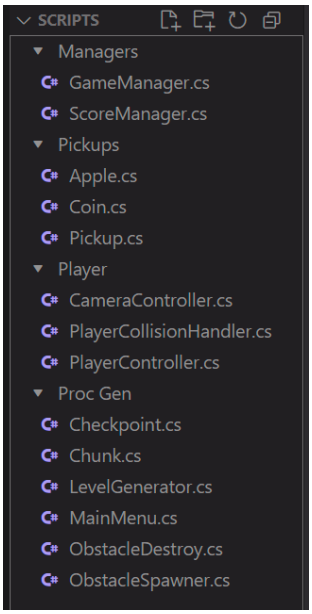
Da bi se osigurala funkcionalnost i preglednost projekta, igra je oblikovana kroz jasno definiranu strukturu. Sustav je podijeljen na više međusobno povezanih cjelina, pri čemu svaka od njih ima svoju ulogu u ostvarivanju konačnog doživljaja igranja. Ovakva podjela omogućuje lakše razumijevanje logike igre, jednostavnije održavanje koda i lakšu mogućnost nadogradnje u budućnosti.

U nastavku se opisuje organizacija projekta u razvojnom okruženju, pregled scena koje čine igru te uloga pojedinih funkcionalnih cjelina.

3.3.1. Organizacija projekta u razvojnom okruženju

Projekt je razvijan u okruženju Unity, koje omogućuje strukturirano organiziranje svih potrebnih elemenata. Pregledna organizacija bila je ključna za uspješno provođenje rada, jer je igra sastavljena od većeg broja komponenti – od trodimenzionalnih modela i animacija do skripti i zvučnih zapisa.

Unutar samog projekta stvorene su mape koje su služile za razdvajanje različitih vrsta sadržaja. Modeli i vizualni elementi bili su smješteni u zasebne mape kako bi bili odvojeni od zvučnih zapisa i glazbene podloge. Skripte su bile organizirane u posebnoj strukturi, pri čemu su one koje se odnose na upravljanje igračem, generiranje prostora, prepreke, prikupljanje predmeta i korisničko sučelje imale vlastite podmape. Na taj je način postignuta jasna preglednost i olakšano je pronalaženje pojedinih dijelova projekta.



Slika 10: Raspored mapiranih C# skripti

Poseban naglasak stavljen je na **modularnost**. Svaka funkcionalna cjelina igre bila je oblikovana kao zasebna komponenta koja se može neovisno uređivati ili nadograđivati. Primjerice, dio sustava zadužen za upravljanje vremenom bio je odvojen od dijela koji upravlja prikazom korisničkog sučelja. Takva modularnost povećava fleksibilnost i omogućuje da se kasnije, bez značajnih promjena u ostatku sustava, dodaju nove funkcionalnosti ili promijene postojeće.

U okviru organizacije projekta osigurano je i praćenje konzistentnog stila imenovanja i dokumentiranja. Time se postiglo da su svi dijelovi igre međusobno usklađeni te da je moguće brzo razumjeti svrhu pojedinih komponenti, što je posebno važno u projektima koji se mogu dodatno razvijati i nakon početne implementacije.

3.3.2. Struktura scena (glavni izbornik i prostor igre)

Projektna struktura igre podijeljena je u dvije osnovne scene koje zajedno čine cjelovito iskustvo igranja: glavni izbornik i prostor igre. Ovakva podjela osigurava jasno razdvajanje funkcionalnosti te pridonosi preglednosti i jednostavnijem održavanju sustava.

Glavni izbornik predstavlja početnu točku igre. Njegova je uloga omogućiti igraču jednostavan pristup glavnim opcijama. U izborniku se nalazi gumb za započinjanje nove igre, gumb za povratak na početni zaslon te opcija za izlazak iz aplikacije. Sučelje je oblikovano jednostavno, bez suvišnih elemenata, jer mu je primarna funkcija navigacija. Na taj način igrač dobiva brz i jasan prijelaz prema glavnom dijelu igre.

Prostor igre čini središnju scenu u kojoj se odvija cjelokupno igranje. U njemu se nalazi lik igrača, okruženje koje se neprekidno nadograđuje, prepreke i predmeti koji se pojavljuju te korisničko sučelje koje prikazuje rezultat, preostalo vrijeme i poruke. Ova scena obuhvaća sve elemente potrebne za ostvarivanje mehanike beskonačnog trčanja.

Slika 11: Glavni izbornik i prostor igre

Podjela na dvije scene omogućila je jasnu granicu između navigacijskog i igranog dijela igre. To ne samo da pridonosi organizaciji projekta, već omogućuje i buduće proširenje, primjerice dodavanje dodatnih scena poput zaslona s postavkama, galerijom rezultata ili uputama za igranje.

3.3.3. Uloga pojedinih funkcionalnih cjelina

Da bi se ostvarilo skladno funkcioniranje igre, cjelokupni sustav podijeljen je na više funkcionalnih cjelina. Svaka od njih ima jasno definiranu ulogu, a njihova međusobna povezanost omogućuje da igra bude dinamična, pregledna i tehnički stabilna.

Prva funkcionalna cjelina odnosi se na upravljanje igračem. Ona obuhvaća kontrolu kretanja, reagiranje na naredbe korisnika te ograničavanje kretanja unutar predviđenog prostora. Istodobno uključuje i reakcije lika na sudare s preprekama, što je ostvareno povezivanjem s animacijskim sustavom. Upravo ova cjelina predstavlja osnovu interakcije igrača s igrom, budući da preko nje korisnik ostvaruje izravnu kontrolu nad tijekovima igre.

Druga funkcionalna cjelina bavi se dinamičkim oblikovanjem prostora igre. Prostor se ne prikazuje u potpunosti unaprijed, već se novi dijelovi postupno pojavljuju ispred lika, dok se već prijeđeni dijelovi uklanjaju. Na taj je način omogućeno neprekidno trajanje igre, a istodobno je smanjeno opterećenje sustava, jer se u svakom trenutku obrađuju samo vidljivi elementi. Ova cjelina odgovorna je za održavanje osjećaja neprekidnog kretanja kroz beskonačan svijet.

Treća funkcionalna cjelina odnosi se na sustav prepreka i predmeta. Ona kontrolira pojavu prepreka koje igrač mora izbjegavati, kao i predmeta koji donose različite učinke - bodove, promjenu brzine ili produženje vremena igranja. S obzirom na to da se prepreke i predmeti pojavljuju u nasumičnim kombinacijama, ova cjelina unosi u igru element nepredvidivosti i sprječava ponavljanje istih obrazaca.

Četvrta funkcionalna cjelina jest praćenje vremena i rezultata. Ona upravlja odbrojavanjem preostalog vremena i obračunom bodova. Završetak igre nastupa kada vrijeme istekne ili kada igrač doživi sudar s preprekom, a konačan rezultat prikazuje se putem korisničkog sučelja. Time ova cjelina osigurava jasan okvir igre i definira njezin završetak.

Peta funkcionalna cjelina obuhvaća korisničko sučelje. Uloga joj je informirati igrača o aktualnom rezultatu, preostalom vremenu i statusu igre. Uključuje i završne poruke, primjerice obavijest o završetku igre. Zbog jednostavnog, ali preglednog dizajna, korisničko sučelje nadopunjuje osnovnu mehaniku i omogućuje igraču donošenje odluka na temelju jasnih informacija.

Šesta cjelina vezana je uz navigaciju kroz scene. Glavni izbornik i prostor igre međusobno su povezani, a igrač jednostavno može započeti novu igru, vratiti se na početni zaslon ili izaći iz aplikacije. Ova funkcionalna cjelina osigurava da igra bude zaokružena cjelina, a ne skup nepovezanih elemenata.

Sve navedene cjeline međusobno djeluju kao jedinstven sustav. Njihova modularna organizacija pridonosi preglednosti projekta i omogućuje buduća proširenja, poput dodavanja novih vrsta predmeta, složenijih prepreka ili dodatnih scena. Na taj način razvijena je čvrsta struktura koja nadilazi okvire samog prototipa i pruža osnovu za daljnji razvoj.

3.4. Implementacija funkcionalnosti

Nakon što su definirana pravila igre, lik igrača, okruženje i osnovna struktura sustava, slijedila je faza implementacije funkcionalnosti. U ovom dijelu prikazuje se na koji su način teorijski koncepti pretočeni u praktična rješenja unutar razvojnog okruženja.

Implementacija je obuhvatila više međusobno povezanih elemenata: upravljanje igračem i sustav sudara, dinamičko oblikovanje prostora igre, dodavanje prepreka i predmeta, praćenje bodova i vremena, upravljanje kamerom te integraciju zvučnih i vizualnih elemenata. Svaka od ovih komponenti imala je posebnu ulogu u oblikovanju cjelovitog doživljaja igranja, a njihovo usklađeno djelovanje osiguralo je da igra bude dinamična, pregledna i izazovna.

U nastavku će svaka od funkcionalnosti biti detaljno obrađena i objašnjena.

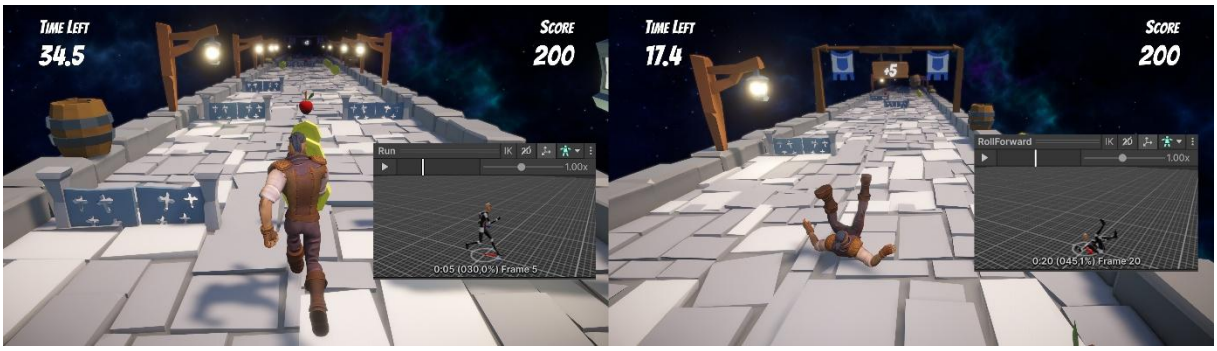
3.4.1. Upravljanje igračem i sustav sudara

Upravljanje igračem predstavlja jezgru interaktivnog dijela igre i najvažniji kanal komunikacije između korisnika i računalnog sustava. Lik kojim igrač upravlja kreće se unutar unaprijed definiranih granica, a igrač putem tipkovnice određuje smjer kretanja. Sustav je oblikovan tako da dopušta pomicanje ulijevo, udesno, naprijed i unatrag, dok mogućnost skakanja nije predviđena. Time je kontrola pojednostavljena, no istodobno zahtijeva brze i precizne reakcije jer se prepreke neprekidno pojavljuju na stazi i približavaju se liku.

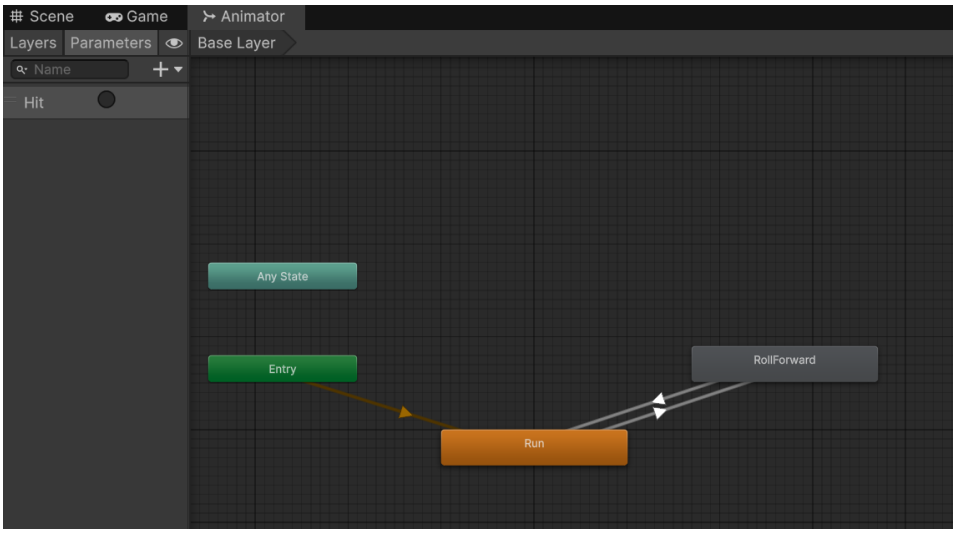
Kontrole su osmišljene tako da budu intuitivne i lako razumljive. Osnovni cilj bio je omogućiti igraču da se može usredotočiti na izbjegavanje prepreka i prikupljanje predmeta, a ne na složene kombinacije tipki. Istodobno, sustav je osjetljiv na vrijeme reakcije: svaka naredba mora biti brzo registrirana i odmah prikazana u igri. Na taj način stvara se dojam glatkog kretanja i izravne povezanosti između korisnikovih postupaka i prikaza na ekranu. Preciznost kontrola pritom je ključna jer se već i najmanje kašnjenje može negativno odraziti na ishod igre.

Za pravilno funkcioniranje fizike i detekcije sudara koriste se osnovne Unity komponente Rigidbody i BoxCollider. Rigidbody omogućuje da se igračev objekt kreće i reagira u skladu s fizikalnim pravilima unutar igre, dok BoxCollider definira granice prostora objekta i registrira kontakte s drugim objektima. Kombinacijom ovih komponenti postiže se realističan sustav kretanja i precizna detekcija sudara između igrača, prepreka i prikupljivih predmeta.

Posebnu vrijednost u sustavu upravljanja ima animacijski modul. **Animator** je zadužen za prikaz odgovarajućih animacija u skladu s postupcima igrača i događajima u igri. Lik je stalno prikazan u animaciji trčanja, čime se naglašava dojam dinamičnosti i stalnog kretanja naprijed. Kada dođe do sudara s preprekom, aktivira se posebna animacija udara, tj. pada. Ova vizualna povratna informacija ima dvojaku ulogu: igrač odmah uočava posljedicu pogreške, a igra dobiva dodatnu dozu realističnosti i izražajnosti.



Slika 12: Animacija trčanja i animacija spoticanja



Slika 13: Animator unutar Unityja

Sustav sudara u ovoj igri ne funkcionira na način da trenutačno prekida igranje. Umjesto toga, sudar je zamišljen kao kazna koja privremeno mijenja ritam igre. Kada lik udari u prepreku, pokreće se animacija udara, a brzina igre se smanjuje. Iako se igrač i dalje može nastaviti kretati, ova promjena ritma dovodi ga u nepovoljan položaj jer raspoloživo vrijeme nastavlja istjecati jednakim tempom. Usporavanje igre znači da se prostor sporije nadograđuje i da je otežano postizanje visokog rezultata, što dugoročno smanjuje šanse za uspjeh.

Na taj način sustav sudara uvodi balans između kažnjavanja i nagrađivanja. Igrač je kažnjen za nepažnju jer se njegov napredak usporava, no nije mu odmah oduzeta mogućnost nastavka. Time se održava tijek igre i smanjuje frustracija, ali se ujedno naglašava važnost preciznog i pažljivog upravljanja.

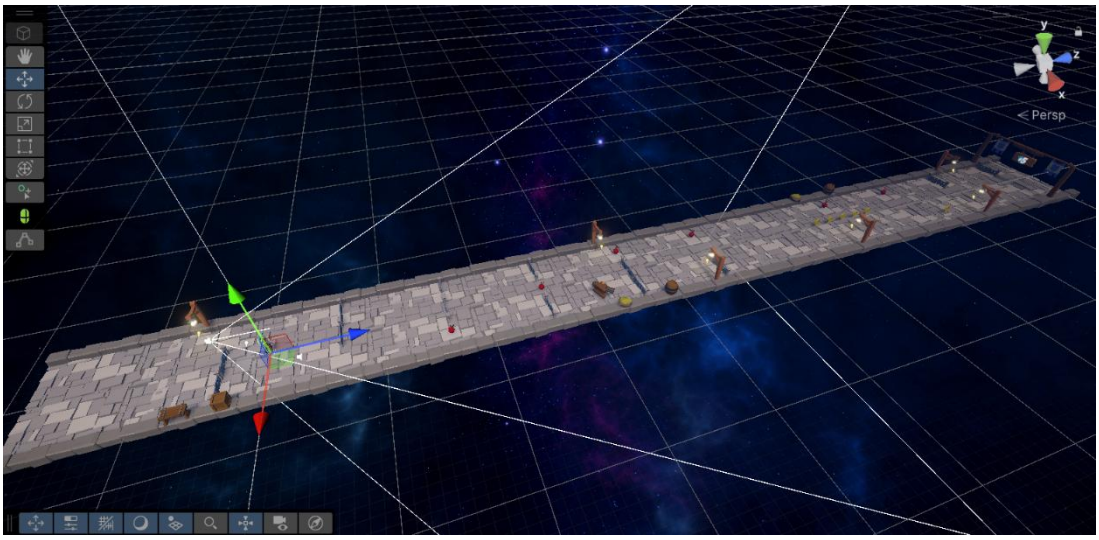
Sudari imaju i širu ulogu jer su povezani s drugim elementima igre. Sustav registrira ne samo dodire s preprekama, nego i kontakt s predmetima i kontrolnim točkama. Kada igrač dotakne predmet, bilježi se povećanje rezultata ili promjena brzine. Kada prođe kroz kontrolnu točku, produžuje se raspoloživo vrijeme. Time sustav sudara ne služi samo za negativne ishode, nego i za prepoznavanje pozitivnih interakcija koje nagrađuju igrača.

Upravljanje igračem i sustav sudara zajedno čine osnovni mehanizam koji definira iskustvo igranja. Oni osiguravaju da korisnik u svakom trenutku osjeti povezanost s virtualnim svijetom, dok Animator i vizualne povratne informacije pridonose dojmljivosti. Sustav sudara pak oblikuje ravnotežu između izazova i mogućnosti nastavka, stvarajući napetost, ali i dopuštajući igraču da ispravi pogreške i pokuša ostvariti bolji rezultat.

3.4.2. Dinamičko oblikovanje prostora igre

Jedna od najvažnijih značajki razvijenog prototipa jest dinamičko oblikovanje prostora igre. Za razliku od tradicionalnih igara u kojima je razina unaprijed definirana i ima jasno određeni početak i završetak, u žanru beskonačnog trčanja prostor se neprekidno nadograđuje ispred igrača. Ovim pristupom osigurava se dojam beskonačnosti i trajna nepredvidivost tijeka igranja.

Prostor igre sastoji se od segmenata koji se automatski dodaju tijekom kretanja lika. Kada igrač napreduje, novi se segment pojavljuje ispred njega, dok se oni iza uklanjaju iz scene. Na taj se način optimizira rad sustava jer se u svakom trenutku obrađuje samo onaj dio prostora koji je vidljiv ili neposredno ispred igrača. Ovakvo rješenje istodobno omogućuje trajnost igre i racionalno korištenje računalnih resursa.



Slika 14: Dinamičko oblikovanje prostora unutar uređivača

Unutar pojedinih segmenata prostora pojavljuju se prepreke, predmeti i kontrolne točke. Njihova raspodjela nije uvijek ista, već se temelji na nasumičnim kombinacijama. Time se izbjegava ponavljanje istih obrazaca i sprječava predvidljivost. Svako novo igranje donosi drukčiji raspored izazova, čime se igraču pruža jedinstveno iskustvo.

Dinamičko oblikovanje prostora ima i važnu ulogu u povećavanju težine igre. Kako vrijeme odmiče, segmenti se pojavljuju brže, a raspored prepreka i predmeta postaje zahtjevniji. Na taj način igrač se suočava s postupnim povećavanjem izazova, što je jedno od osnovnih obilježja žanra beskonačnog trčanja.

Ova funkcionalnost čini osnovu cjelokupnog sustava igre. Bez dinamičkog oblikovanja prostora, igra bi imala ograničeno trajanje i brzo bi izgubila privlačnost. Njegova

implementacija omogućila je da prototip ostane vjeran žanru i da igraču pruži iskustvo koje je istodobno jednostavno za razumjeti, ali izazovno za svladati.

3.4.3. Praćenje bodova i vremena

Praćenje bodova i vremena čini temeljni okvir igre jer određuje njezine ciljeve i granice. Upravo kroz ta dva elementa beskonačno kretanje lika dobiva strukturu i smisao, a igraču se omogućuje jasna evaluacija uspješnosti.

Bodovni sustav temelji se na prikupljanju predmeta u obliku zlatnika. Svaki prikupljeni zlatnik vrijedi točno sto bodova, a zbroj osvojenih bodova neprekidno se prikazuje na korisničkom sučelju. Sustav je oblikovan tako da igrač uvijek ima povratnu informaciju o vlastitom napretku i može pratiti koliko je bodova prikupio. Rezultat se na kraju igre prikazuje u konačnom obliku i predstavlja glavno mjerilo uspješnosti. Time se ostvaruje natjecateljski element koji potiče igrača na višekratno igranje i nastojanje da nadmaši vlastite ili tuđe rezultate.

Vremensko ograničenje uvedeno je kako bi se spriječilo beskonačno trajanje igre i kako bi se osigurao jasan ritam igranja. Na početku svake sesije odbrojavanje započinje od unaprijed definiranog trajanja, koje u ovom prototipu iznosi četrdeset sekundi. Vrijeme se smanjuje u stvarnom vremenu, a preostale sekunde jasno su prikazane na korisničkom sučelju. Kada odbrojavanje dosegne nulu, igra se završava neovisno o tome koliko je bodova igrač prikupio ili koliko je prepreka izbjegao.

Posebnu ulogu imaju kontrolne točke koje se povremeno pojavljuju u prostoru igre. Kada igrač prođe kroz kontrolnu točku, preostalo se vrijeme automatski produžuje za pet sekundi. Ovaj mehanizam daje priliku za dulje trajanje igre i veći broj bodova, ali istodobno zahtijeva dodatnu spretnost jer kontrolne točke nisu stalno dostupne i potrebno ih je dosegnuti unatoč preprekama.

Sustav bodova i vremena usko su povezana dva elementa. Bodovi nagrađuju uspješno kretanje i prikupljanje predmeta, dok vrijeme postavlja granicu i stalno podsjeća na ograničenost trajanja igre. Produžavanje vremena prolaskom kroz kontrolne točke dodaje stratešku dimenziju jer igrač mora paziti na vlastitu poziciju i prilagođavati kretanje ne samo radi izbjegavanja prepreka, nego i radi produljenja trajanja igre.

Na taj način sustav bodova i vremena zajednički oblikuju iskustvo igranja. Oni održavaju ravnotežu između nagrađivanja i izazova, stvarajući okvir u kojem jednostavna pravila igre postaju natjecateljski i motivirajući doživljaj.

3.4.4. Dinamičko upravljanje kamerom

Kamera u računalnim igrama nije samo tehnički element koji igraču prikazuje prostor, već važan alat za oblikovanje dojma i stvaranje atmosfere. U razvijenom prototipu posebna pažnja posvećena je načinu na koji kamera prati lik i reagira na promjene u dinamici igre. Upravo time postiže se uravnotežen spoj funkcionalnosti i estetskog doživljaja.

Za upravljanje kamerom korištena je komponenta **Cinemachine**, dostupna u okruženju Unity. Cinemachine omogućuje dinamičko prilagođavanje kuta gledanja i vidnog polja bez potrebe za složenim ručnim podešavanjem. U igri to znači da kamera u svakom trenutku automatski održava optimalnu poziciju kako bi igrač imao jasan pregled lika i nadolazećih prepreka.

Slika 15: Vidno polje tijekom igranja

Posebna značajka sustava kamere odnosi se na promjene **vidnog polja** (eng. *Field of View, FOV*) tijekom igranja. Kada dođe do ubrzanja igre, kut gledanja se širi i stvara dojam veće brzine i napetosti. Suprotno tome, kada se igra uspori (primjerice nakon sudara), vidno se polje smanjuje, čime se pojačava osjećaj privremenog gubitka ritma. Ove promjene nisu nagle, već se odvijaju postupno, zahvaljujući sustavu glatkih prijelaza koji pruža Cinemachine. Na taj način igrač dobiva jasnu vizualnu povratnu informaciju o promjeni tempa igre.

Slika 16: Promjene vidnog polja tijekom ubrzavanja/usporavanja

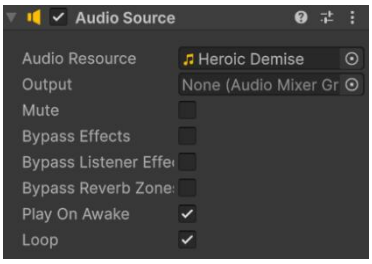
Uz vidno polje, kamera je povezana i s **efektima čestica** koji se aktiviraju pri povećanju brzine. Ovaj vizualni dodatak naglašava dojam ubrzanja i pridonosi osjećaju intenzivnosti igranja. Kombinacija promjena u kretanju kamere i vizualnih efekata čini igru dojmljivijom i uvjerljivijom.

Dinamičko upravljanje kamerom u ovoj igri ima stoga višestruku ulogu. Osigurava preglednost prostora i pravodobno prepoznavanje prepreka, ali istodobno pojačava doživljaj igre prilagođavanjem vizualne perspektive trenutnim događajima. Time kamera prestaje biti pasivan promatrač i postaje aktivan sudionik u oblikovanju iskustva igranja.

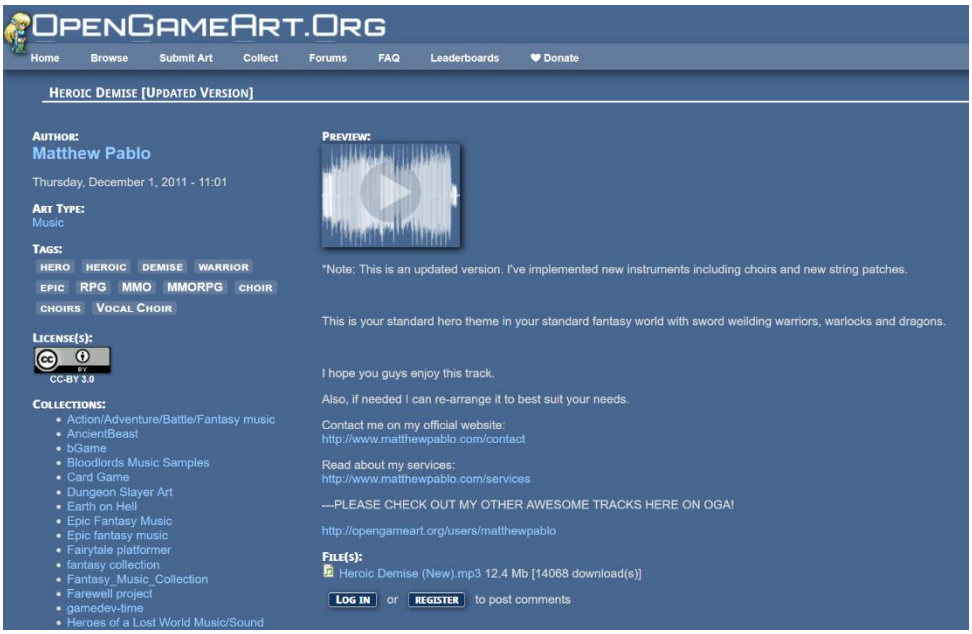
3.4.5. Glazbena podloga

Glazba u računalnim igrama ima ključnu ulogu u oblikovanju atmosfere i pridonošenju ukupnom doživljaju igranja. Čak i kada vizualni elementi i mehanike igre ispunjavaju svoju funkciju, upravo glazba često određuje emocionalni ton i intenzitet iskustva.

U razvijenom prototipu implementirana je pozadinska glazba koja se reproducira u petlji tijekom čitavog trajanja glavne scene. Odabrana skladba nosi naziv *Heroic Demise*, a preuzeta je s platforme OpenGameArt.org. Ova glazba odlikuje se izraženim ritmom i melodijom koja stvara dojam napetosti i bitke. Time se pojačava osjećaj hitnosti, što je u skladu s mehanikom igre u kojoj igrač neprestano trči, izbjegava prepreke i nastoji produljiti svoje trajanje.



Slika 17: Korištena glazba unutar uređivača



Slika 18: Korištena glazba na izvornom mjestu za preuzimanje

Skladba je odabrana i zbog svoje tehničke pogodnosti za petlju. Budući da se igra temelji na beskonačnom kretanju bez unaprijed određenog završetka, pozadinska glazba mora biti u stanju kontinuirano se reproducirati bez prekida i bez stvaranja dojma monotonije. *Heroic Demise* svojim strukturiranim prijelazima i izražajnom dinamikom uspijeva zadržati intenzitet i tijekom duljeg igranja.

Iako se u prototipu koristi samo jedna glazbena podloga, njezina uloga u doživljaju igre vrlo je značajna. Ona naglašava atmosferu neprekidne napetosti i pridonosi uronjenosti igrača u virtualni svijet. Time je potvrđeno da čak i jedan pažljivo odabran zvučni element može imati velik utjecaj na cjelokupno iskustvo igranja.

3.4.6. Navigacija kroz izbornike

Kako bi igra bila cjelovita i pregledna, uz samu mehaniku igranja implementiran je sustav izbornika. Njegova je svrha omogućiti igraču osnovnu navigaciju – od pokretanja igre do izlaska iz aplikacije i povratka na početni zaslon nakon završetka igranja. U prototipu postoje dva ključna izbornika: glavni izbornik i izbornik nakon završetka igre.

Glavni izbornik prikazuje se pri pokretanju aplikacije i sadrži dvije osnovne opcije. Prva opcija jest započinjanje nove igre, čime se igrač odmah preusmjerava u glavni prostor igranja. Druga opcija omogućuje izlazak iz igre. Ovim jednostavnim rješenjem uklonjeni su svi suvišni elementi, a naglasak je stavljen na brzi pristup igranju.

Izbornik nakon završetka igre prikazuje se kada odbrojavanje vremena dosegne nulu ili kada igrač izgubi zbog sudara s preprekom. Na ekranu se tada pojavljuje poruka o završetku igre te jedina opcija – povratak na glavni izbornik. Time se igraču omogućuje da odmah započne novu sesiju ili odluči napustiti aplikaciju kroz opcije glavnog izbornika.

Slika 19: Navigacija kroz izbornike

Navigacija kroz izbornike jednostavna je i intuitivna. Premda je funkcionalnost ograničena na najosnovnije opcije, ona u potpunosti zadovoljava potrebe prototipa. Ovakvo rješenje ujedno ostavlja prostor za buduće nadogradnje, primjerice dodavanje izbornika s postavkama, prikaz najboljih rezultata ili upute za igranje.

3.5. Algoritamska rješenja

Iako se na prvi pogled čini da se igra temelji na jednostavnim pravilima, njezino funkcioniranje u pozadini zahtijeva niz algoritamskih rješenja koja osiguravaju stabilnost, nepredvidivost i trajnost igranja. Upravo algoritmi čine srž sustava i određuju način na koji se prostor igre oblikuje, prepreke i predmeti pojavljuju, a bodovi i vrijeme prate i obračunavaju.

U ovom poglavlju razmatraju se najvažniji algoritamski mehanizmi koji stoje iza igre. Prvo se opisuje dinamičko stvaranje prostora igre, zatim mehanizam pojavljivanja prepreka i predmeta, potom logika praćenja bodova i vremena, a na kraju način postupnog povećavanja težine tijekom igranja. Svaka od ovih cjelina pridonosi cjelovitosti igre i potvrđuje da uspješna implementacija nije rezultat samo vizualnog oblikovanja, nego i pažljivo osmišljene programske logike.

3.5.1. Sustav bodovanja

Jedan od ključnih elemenata svake endless runner igre jest praćenje rezultata igrača. Bodovi stvaraju osjećaj napretka, motiviraju igrača na daljnje pokušaje i omogućuju usporedbu s prijašnjim rezultatima. U ovoj igri sustav bodovanja temelji se na prikupljanju **zlatnika** (eng. *coin*), pri čemu svaki prikupljeni zlatnik nosi **100** bodova.

Sustav je implementiran tako da prilikom kolizije igrača s objektom tipa Coin dolazi do poziva metode za povećanje rezultata u klasi ScoreManager. Ova klasa centralizira logiku bodovanja i automatski ažurira korisničko sučelje. Na taj se način postiže jednostavna i pregledna arhitektura, gdje svaki element igre ima jasno definiranu odgovornost:

- Coin.cs definira vrijednost zlatnika i prosljeđuje zahtjev za povećanje rezultata.
- ScoreManager.cs zbraja bodove i brine da se rezultat odmah prikaže na ekranu.
- Pickup.cs osigurava da se svaki objekt nakon prikupljanja ukloni iz scene i spriječi dvostruko bodovanje.

Ovakav pristup omogućuje lako proširivanje – primjerice, dodavanje novih tipova predmeta koji bi donosili različit broj bodova ili dodatne efekte. Također je osigurana veza s korisničkim sučeljem putem TextMeshPro komponente, što igraču daje stalnu vizualnu povratnu informaciju o napretku.

3.5.1.1. Logika bodovanja

Sustav bodovanja u igri temelji se na jasnom pravilu – svaki prikupljeni zlatnik donosi igraču 100 bodova. Takva jednostavnost omogućava igraču da brzo razumije osnovna pravila i razvije strategiju kretanja kroz stazu. Kod endless runner žanra, ovakav tip nagrade potiče stalni rizik i donošenje odluka: hoće li igrač skrenuti u opasniju traku kako bi prikupio dodatni zlatnik ili će igrati sigurnije i dulje preživjeti.

Logika bodovanja implementirana je u klasi Coin.cs, koja nasljeđuje apstraktnu klasu Pickup. Na taj se način osigurava jedinstven obrazac ponašanja svih predmeta u igri: svaki pickup ima svoj specifičan učinak, ali se nakon prikupljanja uvijek uklanja iz scene. Upravo metoda OnPickup() predstavlja centralno mjesto gdje se definira učinak zlatnika – povećanje rezultata.

Kod u nastavku prikazuje definiciju klase Coin i njezinu osnovnu logiku:

```
using UnityEngine;

public class Coin : Pickup
{
    [SerializeField] int scoreAmount = 100;

    ScoreManager scoreManager;

    public void Init(ScoreManager scoreManager)
    {
        this.scoreManager = scoreManager;
    }

    protected override void OnPickup()
    {
        scoreManager.IncreaseScore(scoreAmount);
    }
}
```

U ovom primjeru vidljivo je nekoliko važnih elemenata. Varijabla scoreAmount označena je kao serijalizirana varijabla tako da se može mijenjati izravno u Unity Inspectoru, bez potrebe za izmjenom koda. Time se otvara mogućnost lakog balansiranja igre – vrijednost se može povisiti ili sniziti ovisno o željenom tempu napredovanja. Metoda Init() koristi se za inicijalizaciju reference na ScoreManager, kako bi se prilikom prikupljanja zlatnika pozvala metoda IncreaseScore().

Metoda OnPickup() je ključna – ona je implementacija apstraktne metode naslijeđene iz bazne klase Pickup. Svaki zlatnik pri prikupljanju poziva scoreManager.IncreaseScore(scoreAmount), čime se rezultat povećava za 100. Ovaj poziv jasno odvaja odgovornosti – Coin je samo zadužen za informiranje da je igrač prikupio bodove, dok ScoreManager centralno upravlja dodavanjem rezultata i njegovim prikazivanjem na korisničkom sučelju.

Budući da Coin nasljeđuje Pickup, naslijeđuje i logiku kolizije koja osigurava da se objekt ukloni nakon što ga igrač prikupi.

To je vidljivo u kodu Pickup.cs:

```
void OnTriggerEnter(Collider other)
{
    if (other.CompareTag(playerString))
    {
        OnPickup();
        Destroy(gameObject);
    }
}
```

U ovom primjeru, čim igrač označen tagom „Player” uđe u koliziju s objektom tipa Pickup, poziva se metoda OnPickup() i objekt se uklanja iz scene. Time se postiže da zlatnik ne može biti prikupljen više puta, čak i ako dođe do višestrukih kolizija u istom kadru igre.

Da bi cijeli sustav bodovanja bio konzistentan i siguran, uvedena je i provjera u klasi ScoreManager. U njoj se prije dodavanja bodova provjerava je li igra završila, što sprečava neželjeno povećanje rezultata nakon *Game Over* stanja. Primjer koda:

```
public void IncreaseScore(int amount)
{
    if (gameManager.GameOver) return;

    score += amount;
    scoreText.text = score.ToString();
}
```

Ovim pristupom ostvarene su tri ključne stvari: prvo, rezultat se ne može povećavati nakon završetka igre; drugo, svaka promjena odmah se prikazuje na ekranu; treće, sva logika bodovanja nalazi se u jednoj metodi, što čini kod preglednim i lakim za održavanje.

Kombinacija klasa Coin, Pickup i ScoreManager pokazuje primjer dobre primjene načela objektno orijentiranog programiranja. Coin je specijalizacija Pickupa i ne zna ništa o detaljima prikaza rezultata. ScoreManager upravlja svim zbrojevima i korisničkim sučeljem, dok Pickup osigurava zajedničko ponašanje svih predmeta u igri. Takva podjela odgovornosti omogućava daljnje proširenje – primjerice, dodavanje novih tipova Pickup objekata koji ne bi povećavali bodove, nego bi možda produžavali vrijeme ili mijenjali brzinu kretanja lika.

3.5.1.2. Povezanost s korisničkim sučeljem

Sakupljanje bodova ne bi imalo smisla kada igrač ne bi imao jasnu i trenutnu povratnu informaciju o svojem napretku. Upravo iz tog razloga, sustav bodovanja u ovoj igri izravno je povezan s korisničkim sučeljem. Svaka promjena u ukupnom rezultatu automatski se prikazuje na ekranu pomoću TextMeshPro komponente. Na taj način igrač je stalno svjestan svoga uspjeha, što povećava motivaciju i dodaje kompetitivnu dimenziju – uspoređivanje rezultata između više pokušaja igre.

Centralna uloga u ovom procesu pripada klasi ScoreManager.cs. Ova klasa ne samo da zbraja bodove već i ažurira UI element koji prikazuje trenutno stanje rezultata. U nastavku je prikazan izvadak iz metode IncreaseScore(), koja se poziva prilikom prikupljanja zlatnika:

```
[SerializeField] TMP_Text scoreText;

int score = 0;

public void IncreaseScore(int amount)
{
    if (gameManager.GameOver) return;

    score += amount;
    scoreText.text = score.ToString();
}
```

Prvi redak pokazuje kako se koristi polje scoreText, serijalizirano pomoću atributa [SerializeField], što omogućava da se komponenta poveže izravno u Unity Inspectoru. Na taj način dizajner igre jednostavno povezuje skriptu sa željenim UI elementom, bez potrebe za dodatnim pronalaženjem objekata putem koda.

U trenutku kada igrač prikupi zlatnik, metoda IncreaseScore() povećava vrijednost varijable score, a zatim ažurira tekstualni prikaz. Linija scoreText.text = score.ToString(); odgovorna je za pretvaranje numeričke vrijednosti rezultata u string te njegovo prikazivanje na ekranu. Time se osigurava da igrač u stvarnom vremenu vidi točan broj bodova koje je prikupio.

Već smo napomenuli i da se prije svake promjene provjerava stanje igre:

```
if (gameManager.GameOver) return;
```

Ova provjera sprečava da se nakon završetka igre rezultat i dalje mijenja, što bi narušilo pravednost sustava bodovanja. Drugim riječima, jednom kada se prikaže *Game Over* ekran, rezultat postaje konačan i više ga nije moguće povećavati.

Povezivanje sa sučeljem realizirano je pomoću TextMeshPro (TMP) sustava. Unity nudi dvije osnovne opcije za prikaz teksta – standardni UI.Text i napredniji TextMeshPro. Odabir TMP-a donosi niz prednosti: bolju kontrolu nad tipografijom, podršku za specijalne znakove, bolju čitljivost i općenito profesionalniji izgled teksta. U kontekstu ove igre, to znači da prikaz rezultata ostaje jasan i pregledan čak i pri velikim promjenama vrijednosti ili većem broju znamenki.

Ovaj način integracije bodova u korisničko sučelje pokazuje kako se jednostavnim rješenjem može ostvariti visoka razina igranosti. Igrač vidi svaku promjenu odmah i na temelju toga može odlučiti hoće li riskirati kako bi prikupio još više zlatnika. Upravo taj osjećaj stalnog rasta rezultata jedan je od ključnih elemenata koji održava napetost i želju za ponovnim igranjem.

3.5.2. Sustav mjerenja vremena

Uz bodove, drugi ključni pokazatelj napretka u igri jest vrijeme. Endless runner igre u pravilu zahtijevaju stalno kretanje igrača, a ograničeno vrijeme stvara dodatni pritisak i osigurava da se svaka partija kad-tad završi. Upravo taj element vremenskog ograničenja doprinosi dinamici igre i tjera igrača na donošenje brzih odluka. Ako bi igra trajala neograničeno, gubila bi na izazovnosti i smislu.

Sustav mjerenja vremena implementiran je unutar klase GameManager.cs, koja centralizira sve logike povezane s trajanjem igre i njezinim završetkom. Na početku svake partije postavlja se početno vrijeme odbrojavanja, koje se zatim kontinuirano smanjuje. Prolaskom kroz kontrolne točke (eng. *checkpoint*) vrijeme se produžuje, čime se nagrađuje uspješno napredovanje. Kada odbrojavanje dosegne nulu, igra se zaustavlja i prikazuje se ekran *Game Over*.

Na ovaj način vrijeme postaje jednako važan resurs kao i bodovi. Dok bodovi motiviraju igrača da riskira i skuplja predmete, vrijeme ga prisiljava da stalno napreduje, jer inače partija završava. Kombinacija ova dva mehanizma čini igru napetom i balansiranom.

3.5.2.1. Početno odbrojavanje

Na početku svake nove igre inicijalizira se određena količina vremena. U skripti GameManager.cs ta vrijednost postavljena je u varijabli startTime. Vrijednost je serijalizirana, što znači da se može jednostavno podešavati iz Unity Inspectora, bez potrebe za izmjenom izvornog koda. Na taj način se lako može prilagoditi težina igre – manja početna vrijednost povećava izazov, dok veća igraču daje više prostora za istraživanje.

Kod u nastavku prikazuje inicijalizaciju vremena:

```
[SerializeField] float startTime = 40f;

float timeLeft;

void Start()
{
    Time.timeScale = 1f;
    timeLeft = startTime;
}
```

U ovom primjeru jasno je da se prilikom pokretanja scene varijabla timeLeft postavlja na vrijednost definiranu u startTime. Također se resetira Time.timeScale na 1, čime se osigurava da igra uvijek krene normalnom brzinom, bez obzira na to je li prethodna partija završila usporeno.

Smanjivanje vremena događa se u metodi Update(), koja se poziva pri svakom okviru prikaza (eng. *frame*) igre. Vrijeme se umanjuje za iznos Time.deltaTime, odnosno proteklo vrijeme između dva framea. Na taj način odbrojavanje je neovisno o performansama računala i uvijek teče stvarnom brzinom. Dio implementacije izgleda ovako:

```
void Update()
{
    DecreaseTime();
}

private void DecreaseTime()
{
    if (gameOver) return;
```

```
    timeLeft -= Time.deltaTime;
    timeText.text = timeLeft.ToString("F1", CultureInfo.InvariantCulture);
}
```

U metodi DecreaseTime() vidi se nekoliko važnih detalja. Prvo, odbrojavanje se zaustavlja ako je igra već završila, kako ne bi došlo do daljnjeg umanjivanja vrijednosti. Drugo, preostalo vrijeme smanjuje se za Time.deltaTime, čime se postiže glatko i precizno mjerenje. Treće, ažurira se korisničko sučelje – vrijednost timeLeft formatira se na jednu decimalu i prikazuje pomoću TextMeshPro komponente. Time igrač u svakom trenutku ima jasan uvid u preostalo vrijeme.

Početno odbrojavanje čini osnovu vremenskog sustava. Bez njega ne bi postojala napetost ni ograničenje koje tjera igrača da konstantno napreduje. Osim toga, ovakav sustav lako se proširuje dodatnim mehanikama, poput kontrolnih točaka koje produžuju trajanje igre ili predmeta koji privremeno mijenjaju brzinu smanjenja vremena.

3.5.2.2. Produženje vremena

Produženje vremena nagrađuje napredak i smanjuje rizik da partija završi prerano. U ovoj igri vrijeme se produžuje na dva načina: prolaskom kroz kontrolnu točku i prikupljanjem posebnog pickupa – **jabuke** (eng. *apple*). U oba slučaja koristi se isti mehanizam, poziv metode u GameManageru koja uvećava trenutno preostalo vrijeme. Na taj način logika ostaje centralizirana i jasna, a sam GameManager ostaje jedina točka istine za trajanje partije.

Temelj funkcionalnosti nalazi se u metodi IncreaseTime unutar GameManager.cs. Metoda prima iznos i dodaje ga na preostalo vrijeme bez dodatnih sporednih učinaka. Takav dizajn je namjeran, jer omogućuje da različiti izvori produciranog vremena (checkpoint, Apple) rade kroz isti ulazni kanal.

Primjer koda u GameManager.cs:

```
public void IncreaseTime(float amount)
{
      timeLeft += amount;
}
```

Checkpoint je najizravniji primjer nagrade za napredak. Kada igrač prođe kroz kontrolnu točku, skripta odgovorna za checkpoint poziva GameManager i dodaje fiksnih 5

sekundi. Ovaj iznos je dio balansiranja težine, dovoljno je velik da nagradi kontinuirani napredak, a dovoljno mali da igrač i dalje osjeća pritisak odbrojavanja.

Primjer poziva iz Checkpoint.cs:

```
void OnTriggerEnter(Collider other)
{
    if (other.CompareTag(playerString))
    {
        gameManager.IncreaseTime(checkpointTimeExtension);
    }
}
```

Uz checkpoint, u igri postoji i pickup tipa Apple, no njegova uloga nije produženje trajanja partije, nego izravno ubrzavanje igre. Kada ga igrač pokupi, Apple poziva metodu ChangeChunkMoveSpeed() u LevelGeneratoru, čime se povećava brzina kretanja staze i proporcionalno prilagođava gravitacija. Time se partija dodatno dinamizira i igrač se izlaže većem izazovu.

Primjer poziva iz Apple.cs:

```
protected override void OnPickup()
{
    levelGenerator.ChangeChunkMoveSpeed(ajdustChangeMoveSpeedAmount);
}
```

Dakle, dok checkpoint pruža klasičnu vremensku nagradu i omogućuje dulju partiju, Apple uvodi element rizika i izazova jer ubrzava tempo igre. Ova dva tipa objekata zajedno tvore ravnotežu između sigurnosti i napetosti – igrač mora odlučiti hoće li igrati na sigurno i tražiti checkpointove ili riskirati s Appleom i time povećati težinu partije.

Važno je napomenuti da metoda IncreaseTime ne ovisi o formatu prikaza niti o stanju korisničkog sučelja. Ažuriranje vizualnog prikaza vremena i dalje se odvija u redovitom toku odbrojavanja (DecreaseTime), gdje se timeLeft svakog framea formatira i prikazuje. Takvo razdvajanje odgovornosti čini sustav predvidivim: komponente koje mijenjaju vrijeme samo mijenjaju vrijednost, a prikaz i završetak partije ostaju pod kontrolom GameManagera.

3.5.2.3. Završetak igre

Kada odbrojavanje dosegne nulu, igra se prekida i prikazuje se završni ekran. Ovu logiku centralno vodi GameManager, koji u sklopu redovite provjere vremena odlučuje je li potrebno zaustaviti igru. Provjera je dio iste funkcije koja svake sekunde smanjuje preostalo vrijeme i osvježava prikaz na zaslonu. Čim timeLeft padne ispod ili do vrijednosti nula, poziva se metoda koja završava partiju. To osigurava dosljedno ponašanje i ubrzava reakciju sustava, bez obzira na brzinu izvođenja.

Ključni uvjet za završetak nalazi se u metodi koja smanjuje vrijeme. U njoj se, nakon ažuriranja prikaza, provjerava je li vrijeme isteklo. Kada jest, poziva se metoda za završetak.

```
if (timeLeft <= 0f)
{
    PlayerGameOver();
}
```

Sama metoda koja završava igru jasno grupira sve posljedice završetka na jedno mjesto. Najprije se postavlja oznaka da je igra gotova, zatim se onemogućuje upravljanje igračem, uključuje se korisnički element s porukom o završetku i usporava vrijeme u sceni. Usporavanje samo naglašava kraj partije i daje igraču trenutak da uoči što se dogodilo, bez ikakvog treperenja ili naglog skoka.

```
void PlayerGameOver()
{
    gameOver = true;
    playerController.enabled = false;
    gameOverText.SetActive(true);
    Time.timeScale = .1f;
}
```

Ovakav pristup odvaja detekciju isteka vremena od posljedica završetka. Provjera ostaje u tijeku odbrojavanja, a sve promjene stanja događaju se unutar jedne, jasno imenovane metode. To pojednostavljuje održavanje, jer se u budućnosti vizualni ili zvučni efekti završetka mogu dodati upravo ovdje, bez mijenjanja ostatka sustava. Budući da je gameOver zastavica postavljena na true, ostatak sustava se odmah prilagođava novom stanju. Funkcije koje inače reagiraju na ulaze igrača ili ažuriraju HUD provjeravaju tu varijablu i prestaju s radom. Time se

izbjegavaju neželjene nuspojave, poput povećavanja rezultata ili ponovnog pokretanja animacija nakon završetka.

Dodatno, na početku nove partije vrijeme izvođenja se resetira na normalnu brzinu. Ta inicijalizacija u startu scene sprječava da nova partija započne usporeno ako je prethodna završila u usporenom modu. Na taj način svaka nova igra uvijek kreće iz istih, očekivanih uvjeta, što je važno za osjećaj pravednosti i konzistentan doživljaj.

3.5.3. Kontrola težine igre

Kontrola težine igre predstavlja ključni element svake endless runner mehanike, jer upravo ona određuje koliko će iskustvo za igrača biti izazovno, a istovremeno i pravedno. Ako je igra prelagana, igrač će brzo izgubiti interes, dok će preteška struktura rezultirati frustracijom i napuštanjem igre. Balans između prepreka, pickupova i brzine kretanja mora se stalno održavati, a u ovoj igri taj je balans ostvaren kombinacijom nekoliko različitih skripti.

Središnji dio sustava težine čini ObstacleSpawner.cs, skripta koja kontrolira pojavljivanje prepreka. Umjesto da se prepreke generiraju nasumično u potpunosti, koristi se korutina koja u pravilnim intervalima stvara nove objekte. Parametar obstacleSpawnTime određuje razmak između instanci, a svako novo pojavljivanje pozicionirano je unutar dopuštene širine staze. Na taj se način izbjegava prenatrpanost i omogućava igraču da razvije ritam izbjegavanja prepreka. Primjer implementacije može se vidjeti u sljedećem isječku:

```
IEnumerator SpawnObstacleRoutine()
{
    while (true)
    {
        GameObject obstaclePrefab = obstaclePrefabs[Random.Range(0,
obstaclePrefabs.Length)];
        Vector3 spawnPosition = new Vector3(Random.Range(-spawnWidth,
spawnWidth), transform.position.y, transform.position.z);
        yield return new WaitForSeconds(obstacleSpawnTime);
        Instantiate(obstaclePrefab, spawnPosition, Random.rotation,
obstacleParent);
    }
}
```

Drugi važan faktor kontrole težine nalazi se u skripti LevelGenerator.cs, koja definira metodu ChangeChunkMoveSpeed(). Ona omogućuje dinamičko povećavanje ili smanjivanje

brzine kretanja levela, ovisno o igračevim akcijama. Ako igrač pokupi Apple, brzina se povećava za unaprijed definirani iznos, dok udarac u prepreku rezultira smanjenjem brzine. Minimalna i maksimalna brzina definirane su unutar skripte, čime se osigurava da igra nikada ne postane prelagana ili neizvedivo brza. Ujedno se prilagođava i gravitacija, čime se održava prirodan osjećaj fizike u igri.

```
public void ChangeChunkMoveSpeed(float speedAmount)
{
    float newMoveSpeed = moveSpeed + speedAmount;
    newMoveSpeed = Mathf.Clamp(newMoveSpeed, minMoveSpeed, maxMoveSpeed);

    if (newMoveSpeed != moveSpeed)
    {
        moveSpeed = newMoveSpeed;

        float newGravityZ = Physics.gravity.z - speedAmount;
        newGravityZ = Mathf.Clamp(newGravityZ, minGravityZ, maxGravityZ);
        Physics.gravity = new Vector3(Physics.gravity.x, Physics.gravity.y,
newGravityZ);

        cameraController.ChangeCameraFOV(speedAmount);
    }
}
```

Treći sloj balansiranja vezan je uz raspodjelu pickupova i prepreka unutar samih segmenata staze. To se ostvaruje u skripti Chunk.cs, gdje metoda SelectLane() određuje raspoložive trake i nasumično postavlja objekte na njih. Na taj se način izbjegava gomilanje više elemenata na istoj liniji kretanja, što bi značajno povećalo težinu i smanjilo pravednost igre.

Sve ove komponente zajedno tvore sustav u kojem težina igre raste prirodno, ali ostaje u granicama igranosti. Igrač osjeća kako se tempo ubrzava što duže traje partija, no istovremeno uvijek postoje mogućnosti za dobivanje dodatnog vremena ili skupljanje bodova. Na taj način stvara se ravnoteža između izazova i nagrade, što je i glavni cilj dobro balansiranog gameplaya.

Sustavi bodova i vremena ključni su za strukturu i napetost igre. Bodovi igraču daju mjerljiv osjećaj napretka, dok vrijeme uvodi ograničenje koje sprječava da partija traje beskonačno. Kombinacija ova dva elementa osigurava da igrač stalno balansira između prikupljanja nagrada i potrebe za preživljavanjem.

Logika bodovanja centralizirana je u klasi ScoreManager, a upravljanje vremenom u GameManageru. Pickup objekti poput coina, checkpointa i jabuke samo prosljeđuju događaje ovim klasama, što arhitekturu čini preglednom i lakom za proširenje.

Balans igre postiže se kontroliranom učestalošću prepreka, prilagodbom brzine i pažljivo definiranim nagradama. Coin donosi 100 bodova, checkpoint produžuje vrijeme za pet sekundi, a Apple ubrzava igru i time povećava izazov.

Ovakav sustav održava igru dinamičnom, pravednom i jednostavnom za nadogradnju, što su osnovna obilježja kvalitetno izvedenog endless runner žanra.

4. Zaključak

Izrada igre tipa endless runner kroz ovaj završni rad pokazala je cjelokupni proces razvoja – od početne ideje i planiranja, preko oblikovanja strukture sustava, do implementacije i testiranja. Teorijski dio rada dao je pregled korištenih alata, principa proceduralne generacije i osnovnih algoritama, dok je praktični dio demonstrirao njihovu primjenu u konkretnom prototipu.

Posebna pažnja posvećena je pravilnoj podjeli funkcionalnosti unutar Unity okruženja. Svaka komponenta igre dobila je vlastitu skriptu i jasno definiranu ulogu, što je osiguralo preglednost i jednostavnije održavanje. Upravljanje igračem riješeno je korištenjem komponente Rigidbody za fizičko kretanje te BoxCollidera za registriranje sudara, dok je LevelGenerator preuzeo zadatak generiranja novih segmenata prostora i održavanja kontinuiteta igre. Mehanizmi prikupljanja predmeta, povećanja bodova i produženja vremena centralizirani su u klasi GameManager, čime je postignuta jasna kontrola toka igre.

Glazba, korisničko sučelje i sustav scena (glavni izbornik i zaslon same igre) zaokružuju iskustvo igranja i pokazuju kako i jednostavan prototip može imati sve osnovne elemente jedne cjelovite računalne igre. Implementacija prepreka kroz kombinaciju BoxCollidera i prilagođenih vrijednosti gravitacije omogućila je da se nadolazeći objekti prirodno gibaju prema igraču, čime je zadržana dinamika i izazovnost igranja.

Rad je pokazao i važnost programskih obrazaca poput nasljeđivanja (klasa Pickup i njezine izvedenice) te korištenja metoda za centralizirano upravljanje logikom. Na taj način dobiven je sustav koji se može nadograđivati – bilo dodavanjem novih vrsta prepreka i predmeta, bilo proširenjem vizualnih i zvučnih elemenata.

Iako je izrađena verzija igre prototip, ona sadrži sve ključne značajke žanra beskonačnog trčanja. Potencijalne nadogradnje uključuju uvođenje višestrukih razina težine, sustava napredovanja igrača, dodatnih vizualnih efekata, te optimizaciju performansi za mobilne uređaje. Takve dorade učinile bi igru dugotrajnijom i atraktivnijom široj publici.

U konačnici, provedeni projekt potvrdio je da Unity predstavlja moćan i fleksibilan alat za razvoj 3D igara, a poznavanje njegovih komponenti i pravilno strukturiranje koda ključ su uspješne implementacije. Izrada igre omogućila je povezivanje teorijskih znanja s praktičnim vještinama, a rezultat je funkcionalan proizvod koji demonstrira osnovne principe dizajna, programiranja i implementacije računalnih igara.

Popis literature

Fandom. (2025). *Minion Rush (2013-2018, 2024-present)*. Preuzeto s https://awesome-games.fandom.com/wiki/Minion_Rush_(2013-2018,_2024-present)

Fandom. (2025). *Subway Surfers Wiki*. Preuzeto s https://subwaysurf.fandom.com/wiki/Subway_Surfers

Gameloft. (2013). *Minion Rush* [mobilna igra]. App Store.

Halfbrick Studios. (2011). *Jetpack Joyride* [mobilna igra]. App Store.

Halfbrick Studios. (2025). *Jetpack Joyride Classic*. Preuzeto s https://www.halfbrick.com/games/jetpack-joyride-classic

Imangi Studios. (2011). *Temple Run* [mobilna igra]. App Store.

Imangi Studios. (2025). *Imangi Studios – Official Site*. Preuzeto s https://imangistudios.com

Jimmy Vegas. (2025). *Unity Tutorials* [YouTube kanal]. Preuzeto s https://www.youtube.com/@JimmyVegasUnity

Microsoft. (2025). *C# Documentation*. Preuzeto s https://learn.microsoft.com/en-us/dotnet/csharp/

Microsoft. (2025). *Visual Studio – Integrated Development Environment*. Preuzeto s https://visualstudio.microsoft.com/

OpenGameArt.org. (2023). *Heroic Demise (Updated Version)*. Preuzeto s https://opengameart.org/content/heroic-demise-updated-version

Stack Overflow. (2025). *Unity game development questions*. Preuzeto s https://stackoverflow.com/questions/tagged/unity3d

SYBO Games. (2012). *Subway Surfers* [mobilna igra]. App Store.

Unity Technologies. (2025). *Cinemachine Documentation*. Preuzeto s https://docs.unity3d.com/Packages/com.unity.cinemachine@latest

Unity Technologies. (2025). *TextMeshPro Documentation*. Preuzeto s https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest

Unity Technologies. (2025). *Unity Asset Store*. Preuzeto s https://assetstore.unity.com/

Unity Technologies. (2025). *Unity Documentation*. Preuzeto s https://docs.unity.com/

Unity Technologies. (2025). *Unity Learn – Game Development Tutorials*. Preuzeto s https://learn.unity.com/

Popis slika

Slika 1: Unity Hub sučelje 3
Slika 2: Unity Asset Store 4
Slika 3: Sučelje igre 9
Slika 4: Model trkača kojim igrač upravlja 10
Slika 5: Modeli prepreka 12
Slika 6: Fizičke postavke za nadolazeće prepreke 13
Slika 7: Pickupi (zlatnik i jabuka) 14
Slika 8: Kontrolna točka 15
Slika 9: Poruka Game Over 16
Slika 10: Raspored mapiranih C# skripti 18
Slika 11: Glavni izbornik i prostor igre 19
Slika 12: Animacija trčanja i animacija spoticanja 21
Slika 13: Animator unutar Unityja 22
Slika 14: Dinamičko oblikovanje prostora unutar uređivača 23
Slika 15: Vidno polje tijekom igranja 25
Slika 16: Promjene vidnog polja tijekom ubrzavanja/usporavanja 26
Slika 17: Korištena glazba unutar uređivača 26
Slika 18: Korištena glazba na izvornom mjestu za preuzimanje 27
Slika 19: Navigacija kroz izbornike 28